ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduction

Information Security

Prof. Stefano Ferretti

Outline

- Part I: Few foundational terms and concepts
- Part II: Some use cases
- Part III: Lessons learned from use cases

Part I: Foundational Terms we Need

Can I give these terms for granted?

- Scalability
- Availability
- Single point of failure
- Bottleneck
- Integrity
- Confidentiality
- Privacy
- Fairness
- Auditability
- Client/Server model
- Peer-to-Peer model
- Churn

Maybe, we can focus on a subset

- Scalability
- Availability
- Single point of failure
- Bottleneck
- **Integrity**
- **Confidentiality**
- **Privacy**
- **Fairness**
- Auditability
- Client/Server model (**some issues only**)
- Peer-to-Peer model
- Churn → refers to the rate at which employees leave an organization. High information security churn causes a loss of institutional knowledge, disrupts security operations, and significantly increases an organization's vulnerability to breaches.

Integrity

Data integrity is the assurance that information remains accurate, complete, and consistent throughout its entire lifecycle

- Data not altered or destroyed in an unauthorized or accidental manner

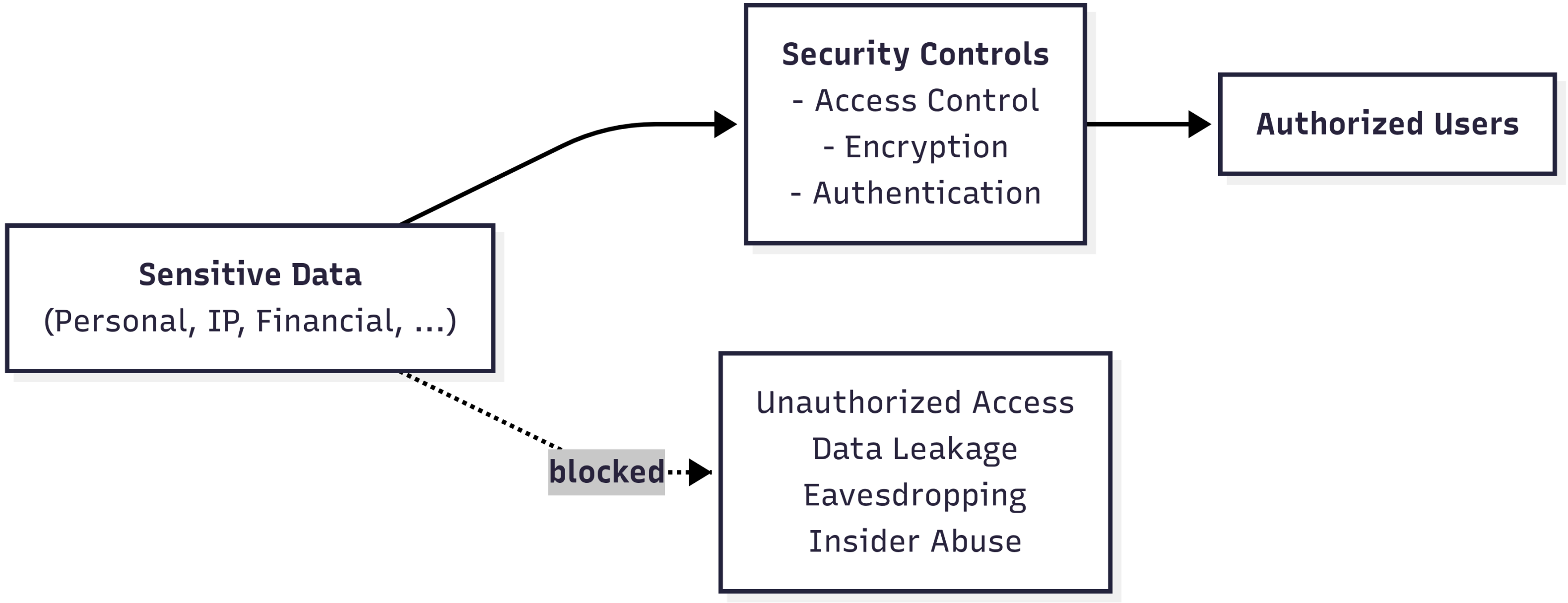
Integrity in ML

- **no data poisoning:** need to ensure training data is not altered
- **model integrity:** need to ensure model parameters are not altered

Confidentiality

Confidentiality ensures sensitive data is accessed only by authorized individuals

- Ensures that only authorized entities can access sensitive information
- Protects data from unauthorized disclosure or exposure



Privacy

Privacy is the right of individuals to exercise control over how their personal data is collected, processed, and shared, ensuring the confidentiality, integrity, and ethical use of their information

- **Privacy in Storage:** store only necessary data (**data minimization**), properly encrypted (**encryption at rest**)
- **Privacy in ML:** individuals not re-identified from model outputs (**membership inference protection**)

A Membership Inference Attack is a specific type of privacy attack in the field of machine learning.
- How the attack works: an attacker takes a record of an individual (e.g. Mario Rossi) and feeds it into the model. The attacker analyses the model's response (often by looking at the prediction confidence scores). Since models tend to respond much more accurately to data they have already seen during training than to entirely new data, the attacker can determine with a very high degree of certainty whether Mario Rossi's data was part of the training dataset.
- Why is this serious? Knowing that an individual is part of the dataset can, in itself, constitute a very serious privacy violation. For example: if the model in question was trained exclusively on patients with a specific rare disease, discovering that Mario Rossi's data was used for training automatically reveals that Mario Rossi has that disease. Protecting against this attack means ensuring that the model's behaviour is statistically identical whether an individual's data was included in the training or not.

Privacy vs Confidentiality

- **Confidentiality → access control**
- **Privacy → inference control**

Inference control analyzes and restricts the information that a user can deduce (infer) by querying a system or observing its outputs.

Normally, to protect sensitive data, you use access control: if a user is not authorized to view a specific file or piece of data, the system blocks them. The problem arises when a malicious user does not attempt to read the restricted data directly, but instead asks legitimate questions or analyzes the system's responses to guess (infer) what they are not supposed to see. Inference control is designed precisely to prevent this from happening, by monitoring or altering public results to ensure that no one can reconstruct the "secret" hidden behind the scenes

Fairness

Fairness ensures that information systems and algorithms operate without creating or reinforcing bias. It implies the fair equitable treatment of all users and data subjects, preventing discriminatory outcomes and ensuring balanced resource allocation across the network

- **Fairness in distributed systems**: fair resource allocation, laod balancing, latency equity
- **Fairness in ML**: bias mitigation, group fairness, algorithmic transparency

The problem of trust

Central Trusted Authority

In the traditional model of computer science (e.g., the classic client-server paradigm), security relied on the presence of a central entity that everyone trusted implicitly (the Central Trusted Authority). Examples: A bank's central server or a hospital's central database.

What we **trust**

- **services**
- **data sources**
- **data**

Under the protective wing of this central authority, traditional systems implicitly assumed that three key elements were always reliable and free from malicious manipulation:
- Services: it was taken for granted that software and algorithms performed calculations correctly, honestly, and without any anomalous behaviour.
- Data sources: it was assumed that those providing the data (e.g., official sensors, company employees, internal databases) were legitimate, authenticated, and non-malicious entities.
- Data: it was assumed that the data entered into the system was clean, accurate, and free from alterations inserted by third parties.

However, in modern systems, **trust cannot be assumed**

In today's systems, nothing can be trusted a priori:
- Unverified data sources: if data is collected collaboratively from thousands of smartphones belonging to unknown users (e.g., traffic data or GPS tracking), you can no longer assume that the data sources are trustworthy. A malicious user could send fake data to compromise the system (Integrity breach).
- The AI supply chain: When training a machine learning model, data is often labelled by third-party companies or models are updated in a decentralized manner (as in Federated Learning). This means an attacker can corrupt the underlying data or send malicious updates directly during training (Data Poisoning and Label Flipping attacks).
- The intrinsic vulnerability of aggregated data: even if the sources appear anonymous, the combination and aggregation of massive amounts of modern data (using OSINT) allows attackers to perform advanced correlations, violating privacy through inference (as demonstrated by the historic Netflix dataset).

Two Fundamental Objectives

Systems should be analyzed along two axes (in addition to ~~besides~~ **performance**):

- **Trustworthiness**

  → Can we rely on data, models, and system behavior?

In modern systems, data comes from unverified sources (e.g., users' smartphones, IoT sensors, the web), and decision-making models (ML) are complex black boxes. Ensuring trustworthiness means implementing mechanisms to ensure that:
- The data is intact: No one has manipulated or altered it at the source (preventing attacks such as data poisoning).
- The models are robust: The algorithm makes correct decisions and is not easily fooled by malicious inputs (preventing evasion attacks or adversarial examples).
- The behavior is predictable: The system does exactly what is expected, without dangerous emergent behaviors or exploitable logical flaws.

- **Privacy**

  → Can we use data without exposing sensitive information?

Modern systems have a hungry need for data to function properly (the more clean data there is, the more AI learns). The Privacy axis focuses on ethical and legal constraints: how can we extract value, statistics, and patterns from this massive amount of data without ever violating the anonymity or rights of the individuals who generated that data? As we saw earlier, this requires controlling inferences (preventing the output from being traced back to an individual).

These are **not independent** and often introduce trade-offs

Increasing security in one area can lower the security of the other. You cannot have the best of both worlds without paying a price. Here are the three main trade-offs between Trustworthiness and Privacy:
1) The Control Trade-off (Verification vs. Anonymity)
To increase Trustworthiness: If I want to be 100% sure that a data source is reliable and isn't sending false data (e.g., fake reviews, altered GPS coordinates), the most logical approach is to identify, track, and audit the data sender. Knowing exactly who sent what destroys the user's privacy. Conversely, if I guarantee total anonymity to protect privacy, I open the door to malicious actors who can send junk data without being detected, undermining the system's trustworthiness.
2) The Trade-off of Statistical Noise (Differential Privacy vs. Accuracy)
To Increase Privacy: To protect ML models from privacy-violating attacks (such as membership inference), we use Differential Privacy, which introduces "random noise" into the data or the model's calculations. If I introduce too much noise, the model becomes less accurate, less stable, and potentially less robust. If a medical model misdiagnoses a patient because we introduced too much noise to protect patient privacy, the system is no longer trustworthy.
3) The Transparency Trade-off (Explainability vs. Vulnerability)
To increase Trustworthiness: A system is considered trustworthy if it is transparent: that is, if the user or certifier can understand why the model made a certain decision (Explainable AI). The more details and explanations the model provides about its internal workings or the reasoning behind its response, the more information it is giving away to an attacker, who can use those explanations to reconstruct sensitive training data (Model Inversion) or to bypass the model (Evasion).

Trust vs Privacy: A Tension

Improving one objective may weaken the other

Examples:

1 - Logging improves **auditability** but may reduce **privacy**
- Data sharing improves **utility** but increases **exposure risk**

2 - Differential Privacy improves privacy but reduces model accuracy

Does the AI revolution change the overall picture?

In terms of trustworthiness, privacy, scalability, fairness, auditability, etc.

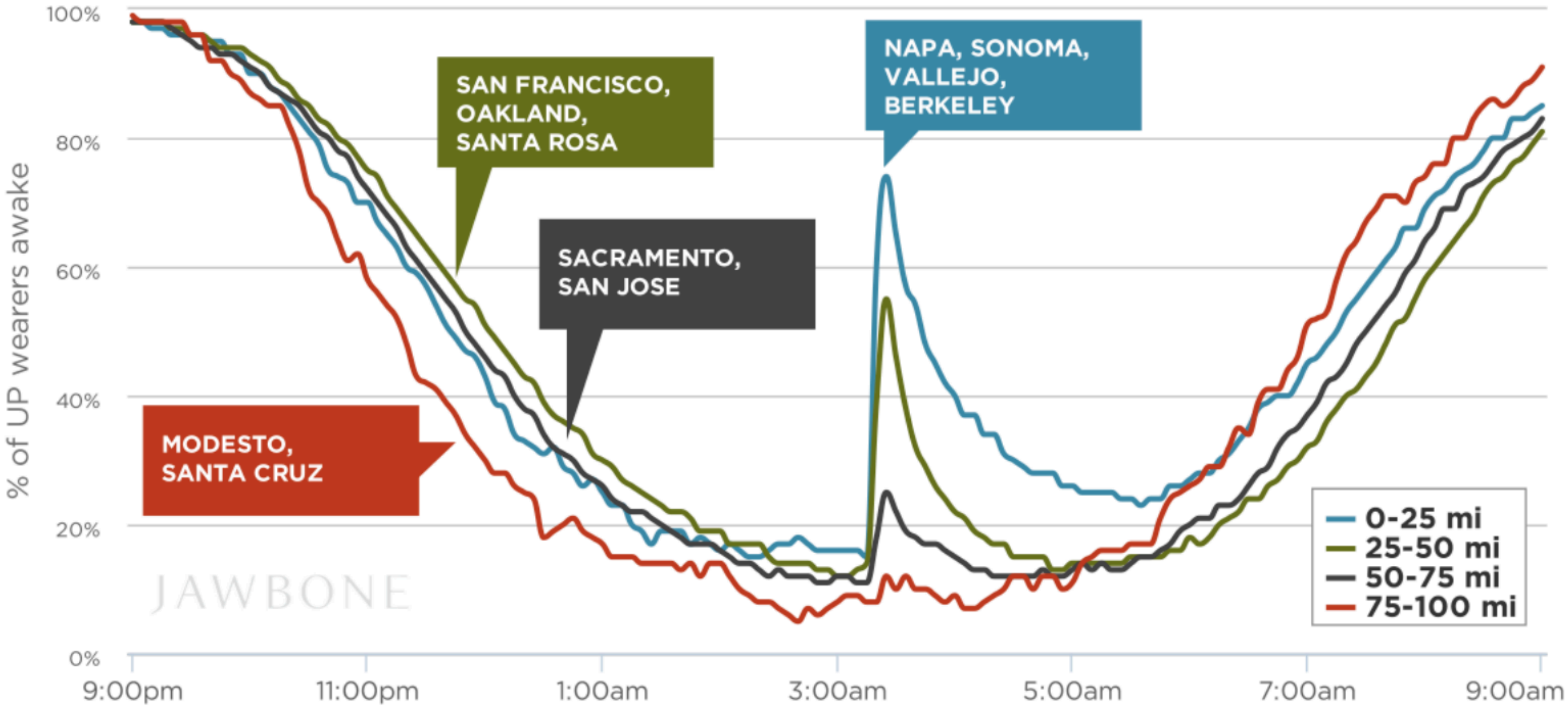
Does the AI revolution change the overall picture?

In terms of trustworthiness, privacy, scalability, fairness, auditability, etc.

Trustworthy AI

Designing and ensuring Trustworthy AI means creating Artificial Intelligence systems that humanity can rely on safely and ethically, even in the presence of hostile environments or malicious attackers. A Trustworthy AI must simultaneously meet four fundamental requirements:
- Adversarial Robustness: the model must be mathematically and statistically robust against manipulation attempts. It must be able to withstand attacks both during the training phase (using Robust Aggregation Rules to neutralize poisoned updates) and during the inference phase (using Adversarial Training to avoid being fooled by adversarial examples).
- Certified Mathematical Privacy: a Trustworthy AI must incorporate advanced protections such as Differential Privacy, introducing calibrated noise into calculations to mathematically guarantee that the model's output never reveals whether a single individual's record was used or not.
- Data Supply Chain Governance and Control: since AI is vulnerable to corrupted data, ensuring reliability requires strict control over the data lifecycle. For example, risks associated with external suppliers labelling data can be mitigated by implementing redundant checks (e.g., having the same record labelled by multiple independent annotators and measuring the degree of agreement).
- Emergent Security in Distributed Systems: when multiple AI agents interact with one another in a decentralized manner, the system becomes dynamic and unpredictable. In this context, a Trustworthy AI must be able to manage trust dynamically and rely on engineered network structures (Topology) to prevent the compromise of a single agent from causing the collapse or misinformation of the entire network.

The landscape has changed



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The more data you collect from various sources, the more your knowledge grows. A single piece of public data on its own means nothing, but millions of data points together create value.
Correlation & Patterns: By combining different data, algorithms and analysts can identify stable correlations and recurring patterns that were previously invisible.
Insights: From these correlations, deep insights, predictive models, and strategic decisions are derived.

The landscape has changed

The more data is publicly available, the larger the attack surface becomes, and the more people (or organizations) are exposed.
Inference: An attacker can use seemingly harmless pieces of information and, by combining them, deduce (infer) a secret that was never publicly disclosed.
Re-identification: Even if a company publishes an anonymized dataset, an attacker can take that dataset and cross-reference it with other OSINT sources. Through this correlation, the attacker can uncover the real identity behind that anonymous data.
Hidden sensitivity: Data that today seems trivial and non-sensitive (e.g., your running routes tracked by an app), when aggregated with other data, can reveal highly sensitive information.

Open Source Intelligence (OSINT)

More data → More knowledge

- Correlation
- Patterns
- Insights

Aggregation is power

More data → More exposure

- Inference
- Re-identification
- Hidden sensitivity

Aggregation is risk

Real-world example

Netflix Prize Dataset (2006)

- "Anonymized" movie ratings released
- No names, no direct identifiers

↓

- Cross-referenced with IMDb ratings
- Users re-identified

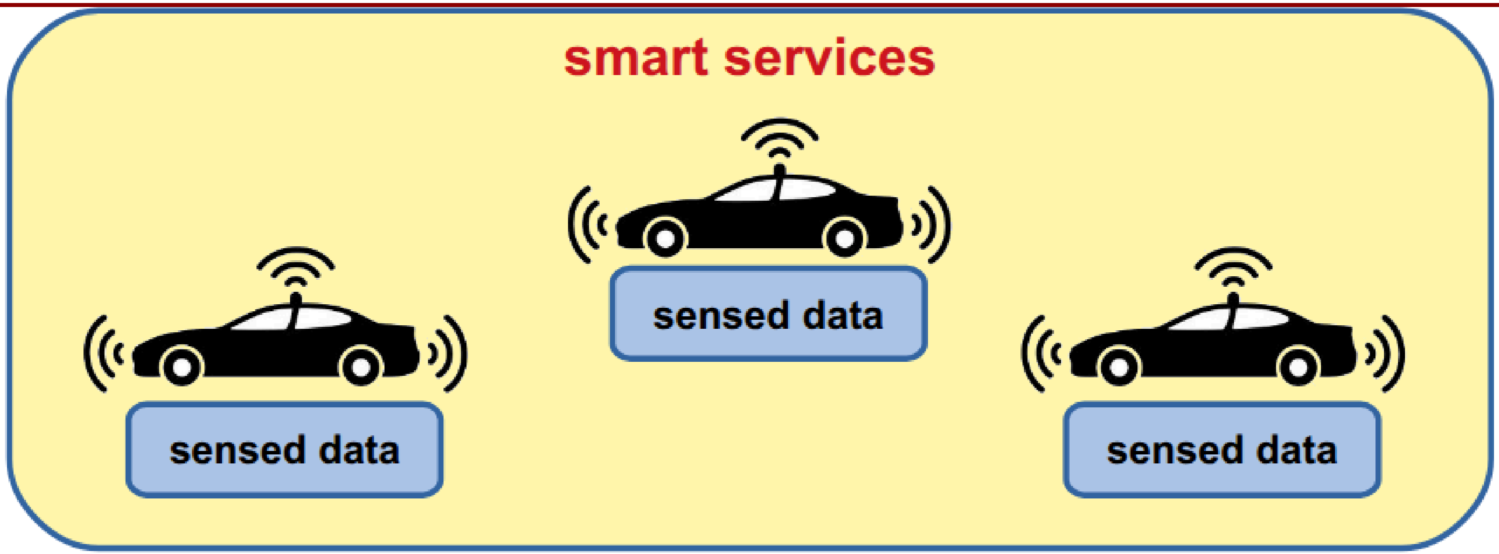
The Netflix dataset was cross-referenced with public data extracted from IMDb (Internet Movie Database), a well-known online platform (and source of OSINT) where users review and rate movies in a completely public manner, signing in with their first and last names or a traceable nickname.

Part II: Some Use Cases

Use Case #1: Crowdsensing Applications

(in Decentralized Systems)

Smart Transportation Systems



Smart Transportation Systems

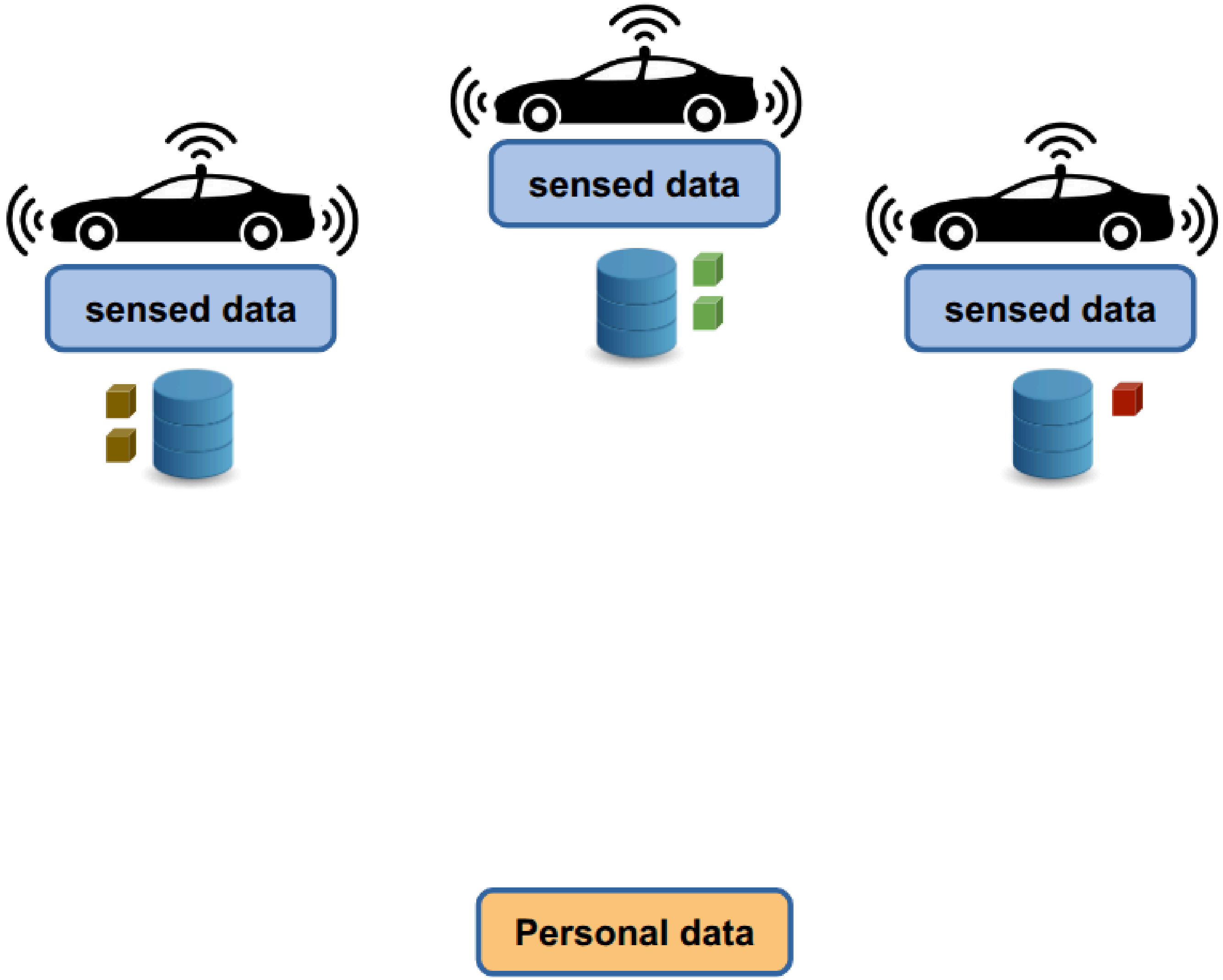
The system implements **crowdsensing-as-a-Service**

Crowdsensing is a paradigm in which a multitude of ordinary users, using sensors built into their everyday mobile devices (such as smartphones, smartwatches, or in-car computers equipped with GPS, accelerometers, cameras, etc.), collect and share data useful for monitoring a phenomenon of common interest. When this paradigm becomes as-a-Service, it means that a decentralized technological infrastructure is established to continuously collect, aggregate, and process this mass data in order to resell it or offer it in the form of application services to users or companies.

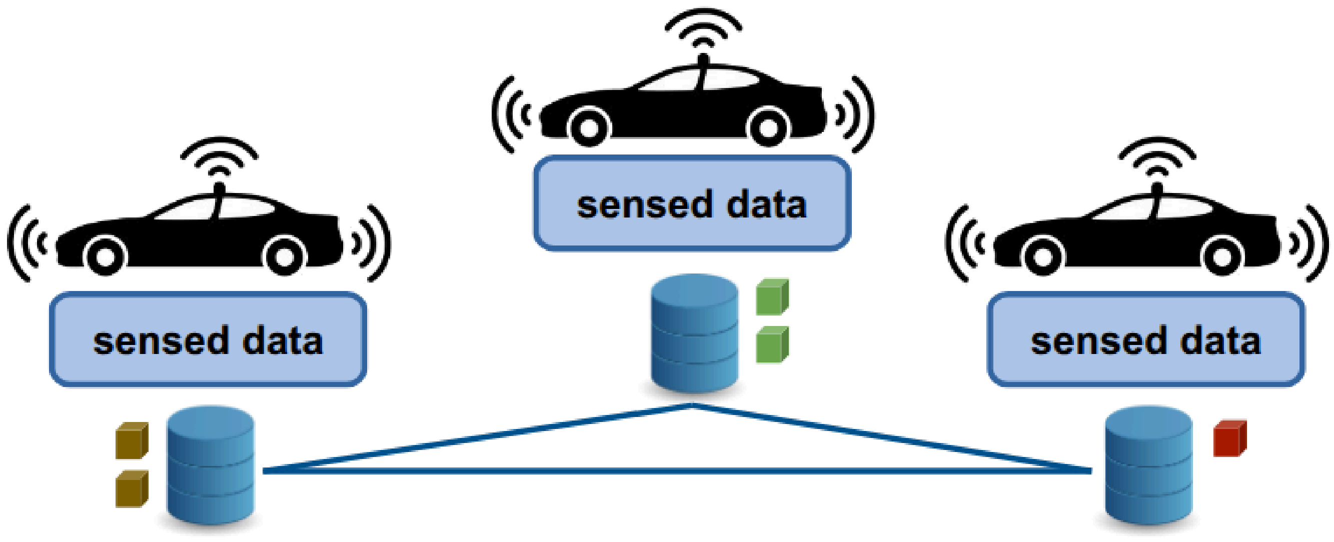
Data used to provide:

- Traffic analysis
- Route optimization
- Safety alerts (e.g., crash, wrong-way driving, weather conditions)

Smart Transportation Systems



Smart Transportation Systems



Data storage

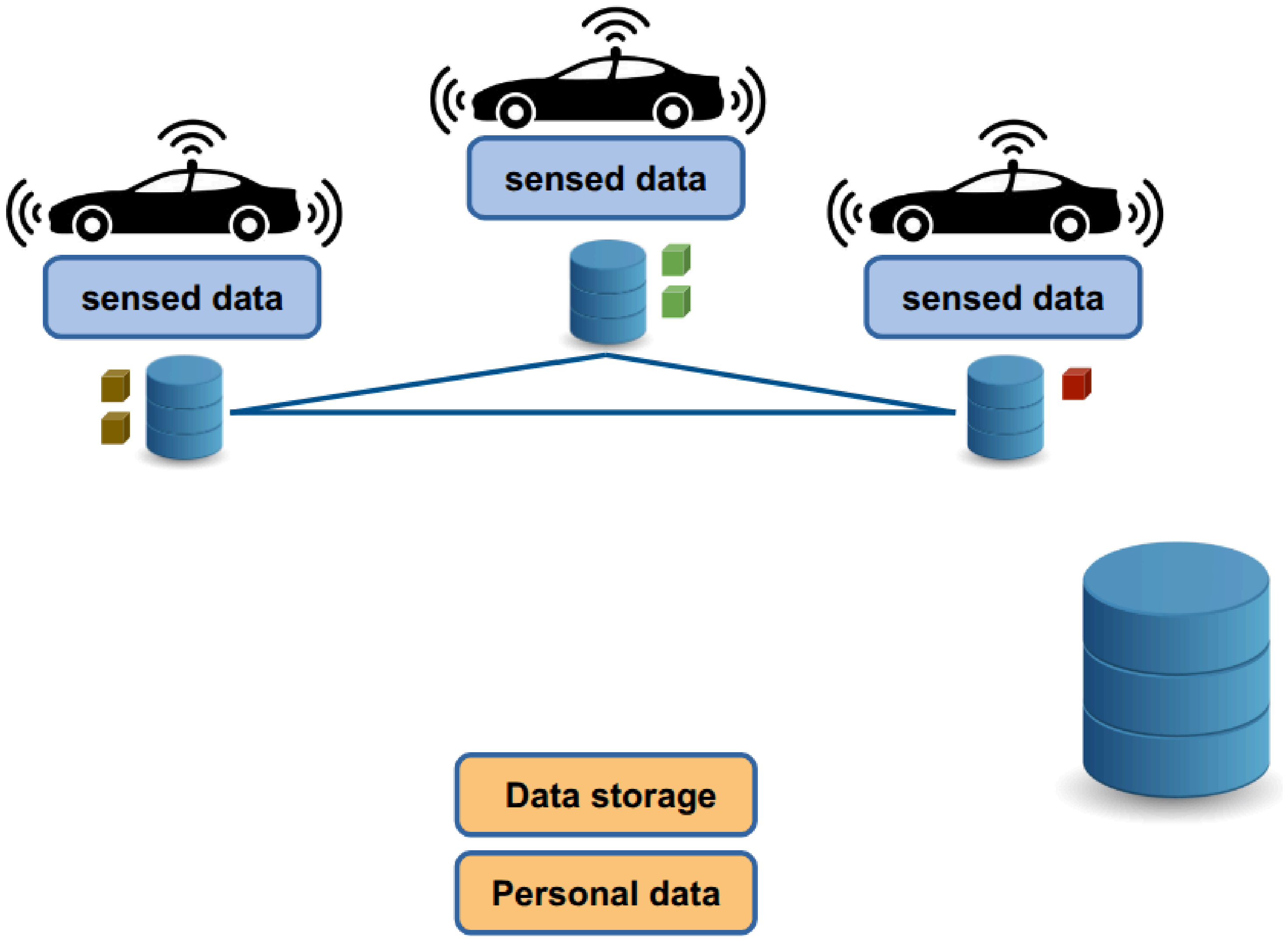
Personal data

Security Objectives

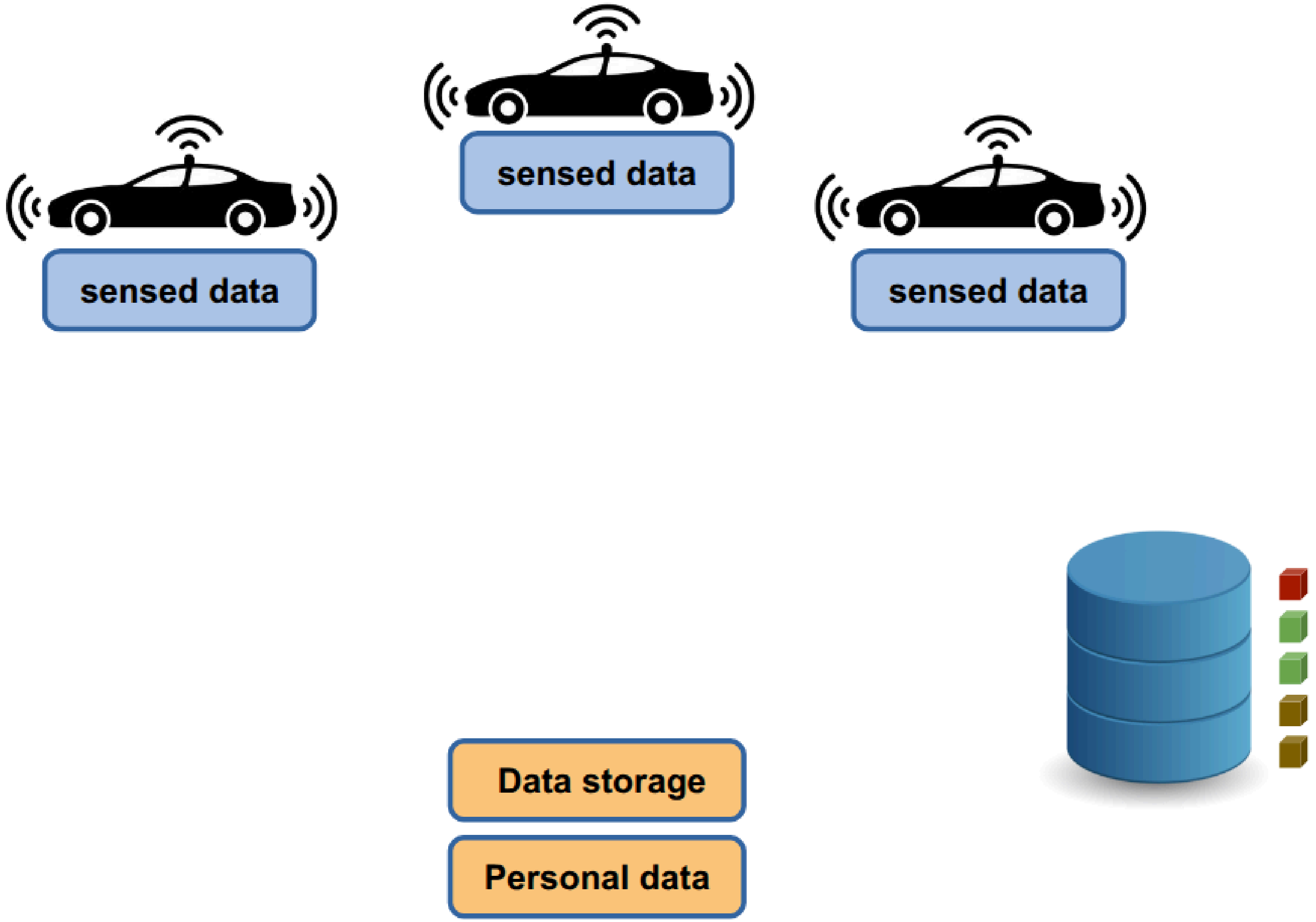
The system must guarantee:

- **Integrity** → Data cannot be altered undetectably
- **Confidentiality** → Unauthorized parties cannot access raw data
- **Availability** → Data remains retrievable when needed
- **Authenticity** → Data provenance is verifiable
- **User Sovereignty** → Data subjects retain control

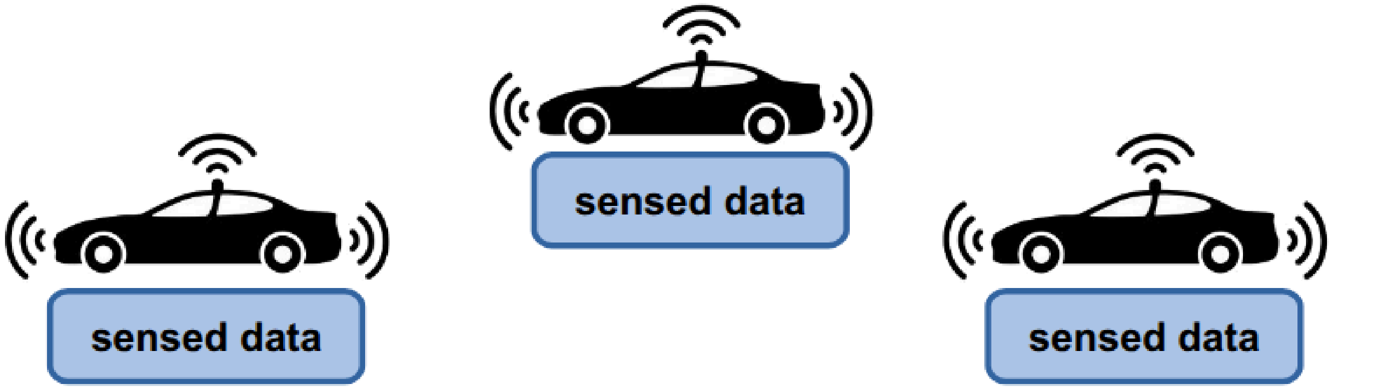
Opt1: central entity maintains crowdsourced data



Opt1: central entity maintains crowdsourced data



Opt1: central entity maintains crowdsourced data



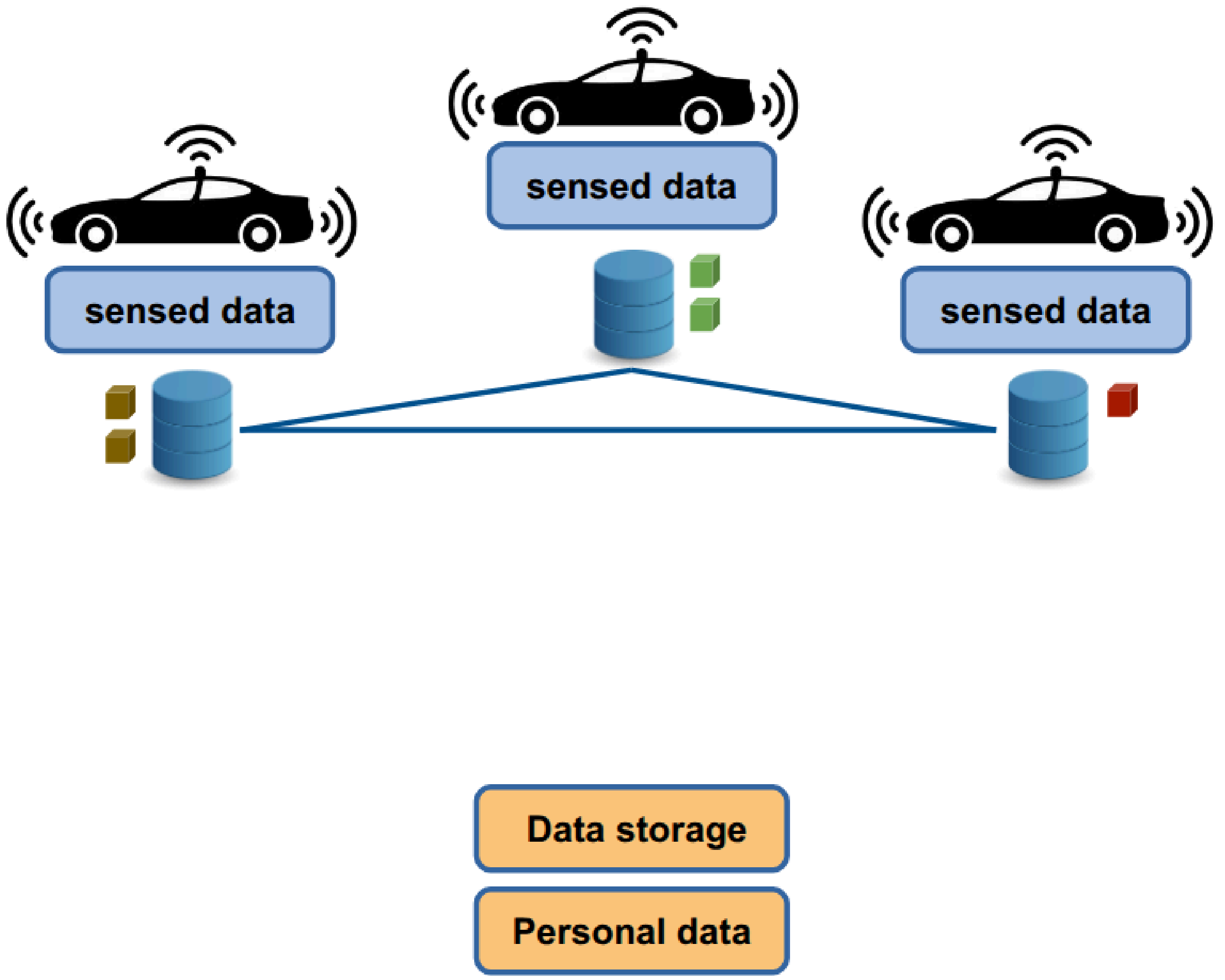
Option 1 cannot be implemented because it does not respect the principle of user sovereignty

Users lose the sovereignty over their data
- data controller gains all the data
- data controller can alter the data
- users must rely on the controller

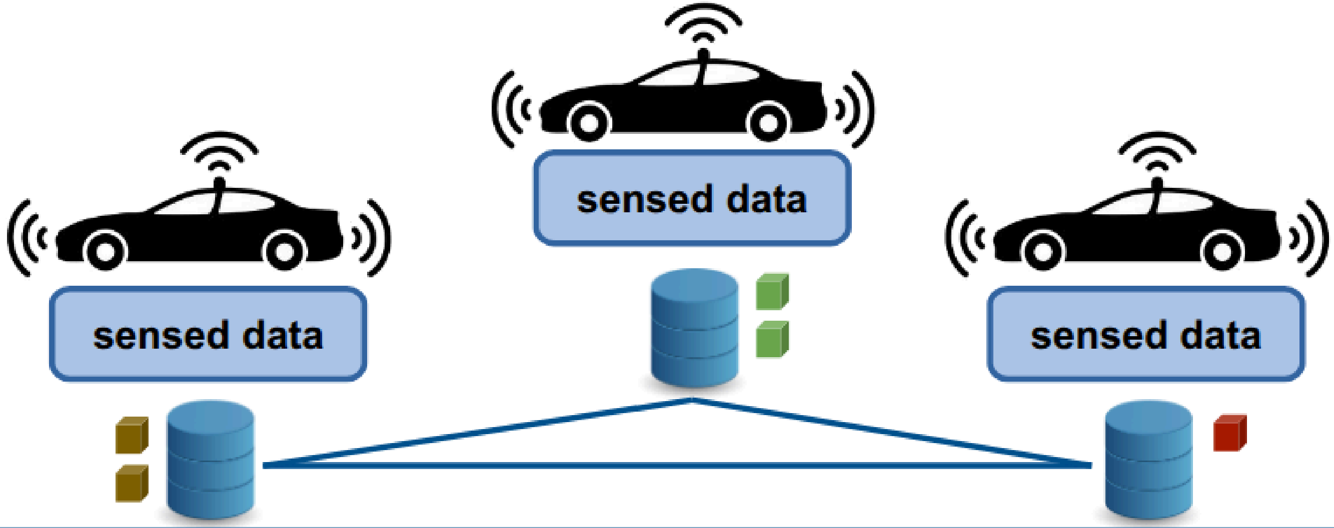
Data storage

Personal data

Opt2: keep data locally - distribute upon request

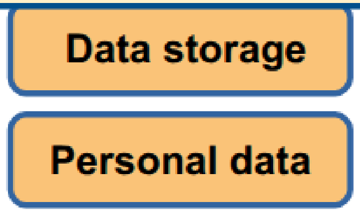


Opt2: keep data locally - distribute upon request



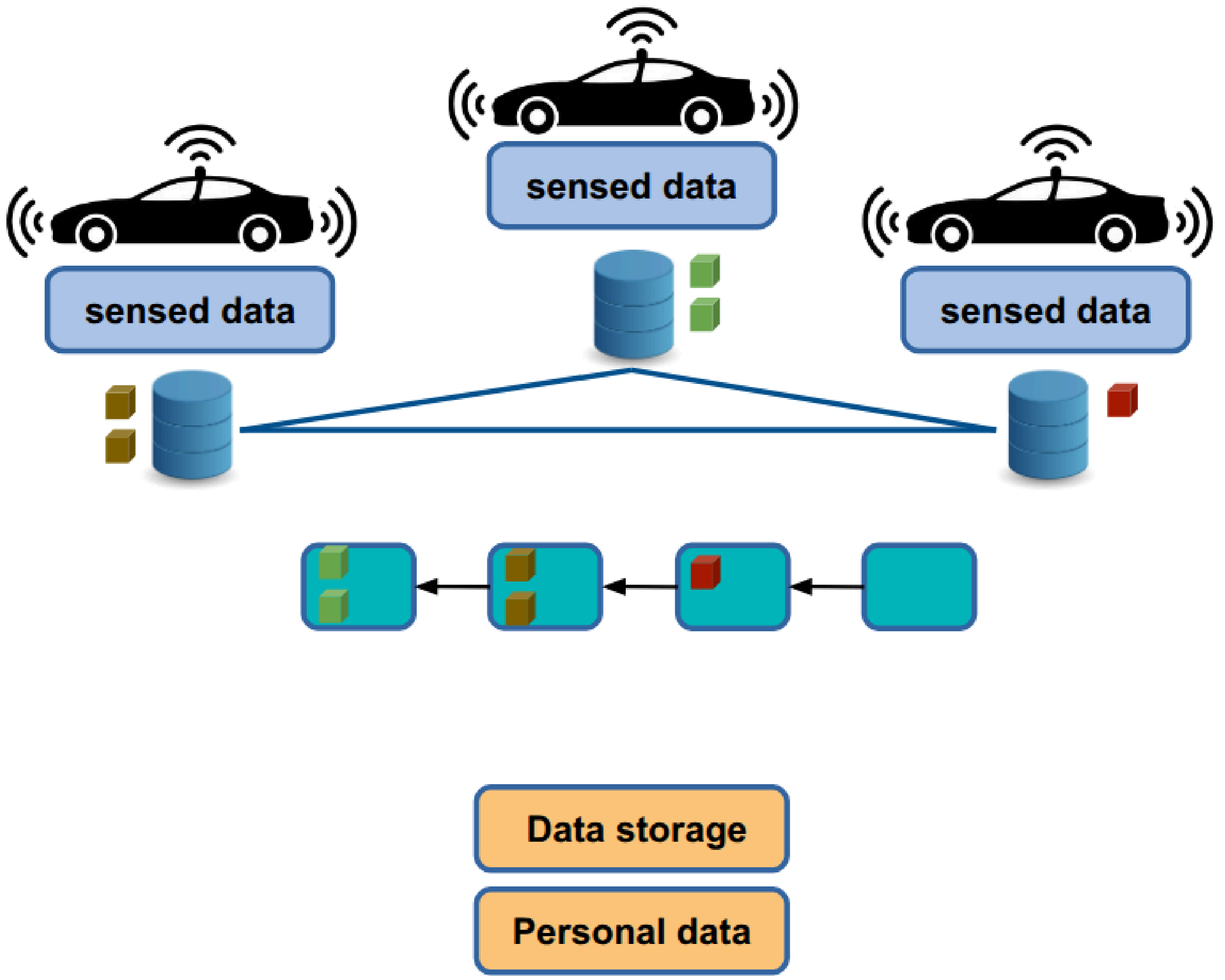
Pros
- you maintain your own data

Cons
- not practical, you need to be always reachable
- storage, computation, communication capabilities needed

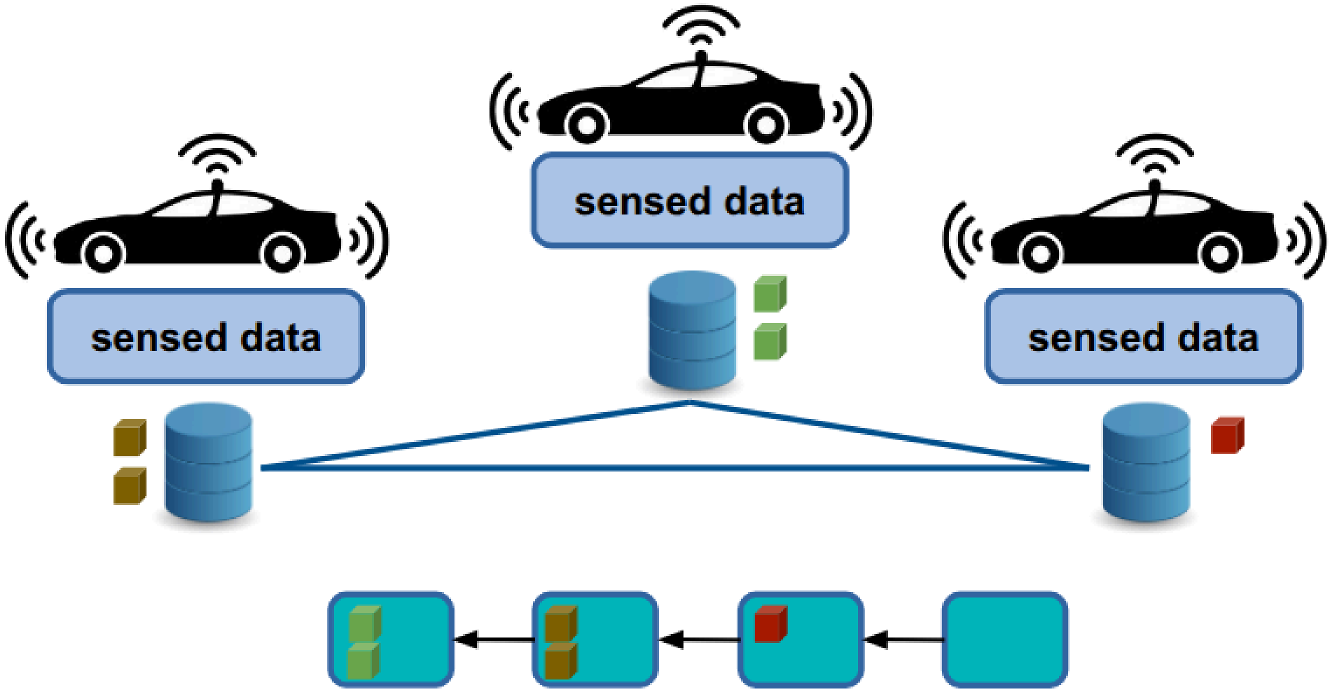


Opt3: use a ledger to register data

A ledger is a shared, decentralized digital registry of data. Rather than being stored on a single centralized server, it is replicated and synchronized across a network of independent nodes. A ledger is based on two fundamental security properties:
- Append-only: new data can only be added sequentially to the end of the ledger; existing entries cannot be modified or deleted.
- Immutability (tamper resistance): thanks to cryptographic mechanisms, once data is recorded, it becomes mathematically impossible for any entity to alter or falsify it retroactively without being immediately detected.



Opt3: use a ledger to register data



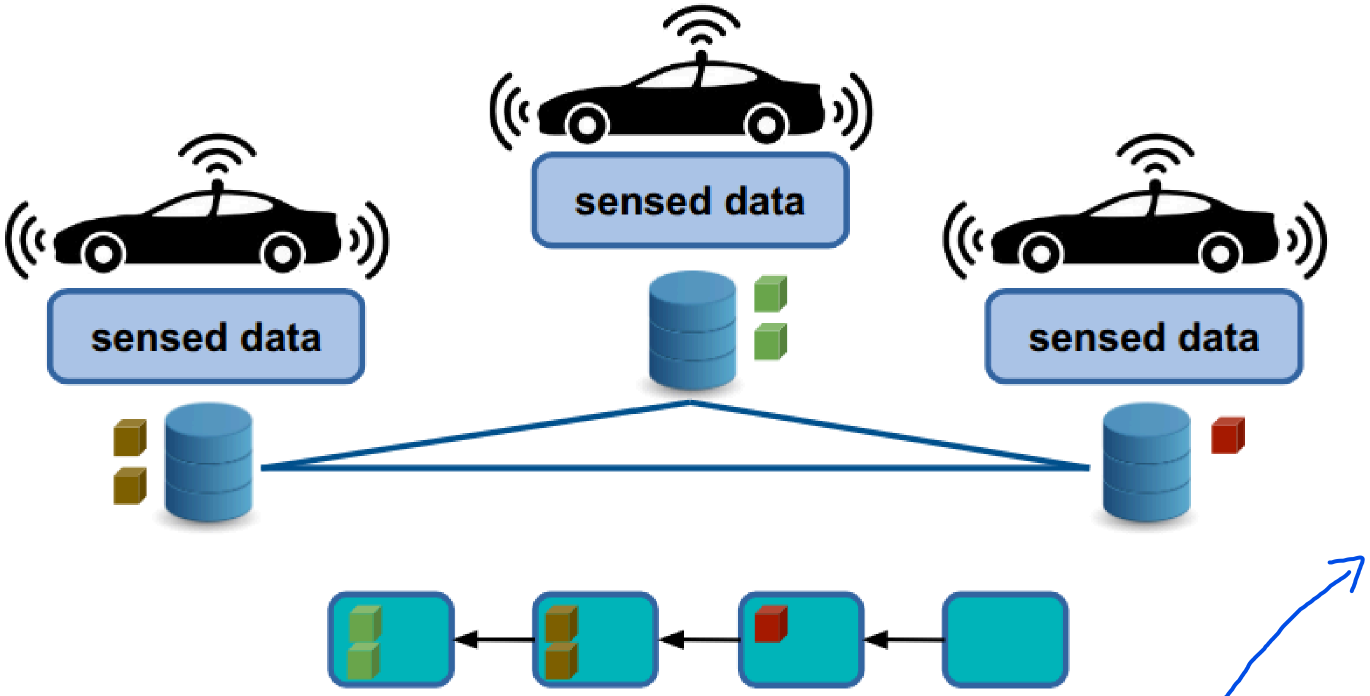
Pros:
- rather simple
- data integrity
- traceability

Data storage

Personal data

Opt3: use a ledger to register data

- Simple: the architecture relies on an existing, standardized ledger infrastructure. The system simply needs to send transactions in append to the ledger.
- Data Integrity: once the data is written to the ledger, it can no longer be altered or retroactively deleted by anyone. This neutralises attacks at their source where an attacker attempts to tamper with or falsify historical mobility data.
- Traceability (Verifiability): every single data entry in the ledger receives an immutable timestamp and is digitally signed. So, anyone with the necessary permissions can verify the exact chronology of the data entered from its source, ensuring the authenticity of its origin.



- Only suitable for small amounts of data: writing information "inside" the blocks of a distributed ledger requires computation, network consensus, and storage space duplicated across many nodes. Consequently, storing large amounts of data is economically unsustainable and technically prohibitive.
- No right to be forgotten/rectified: it is the greatest trade-off between integrity and privacy. Under current regulations (such as the GDPR), users have the legal right to request the deletion of their personal data. However, the fundamental property of the Ledger is perpetual immutability. If sensitive or identifying data ends up on the ledger by mistake, it will remain there forever, violating the principle of User Sovereignty.
- Latency: a decentralized ledger requires network nodes to reach algorithmic consensus to confirm a transaction. This introduces delays (latency).

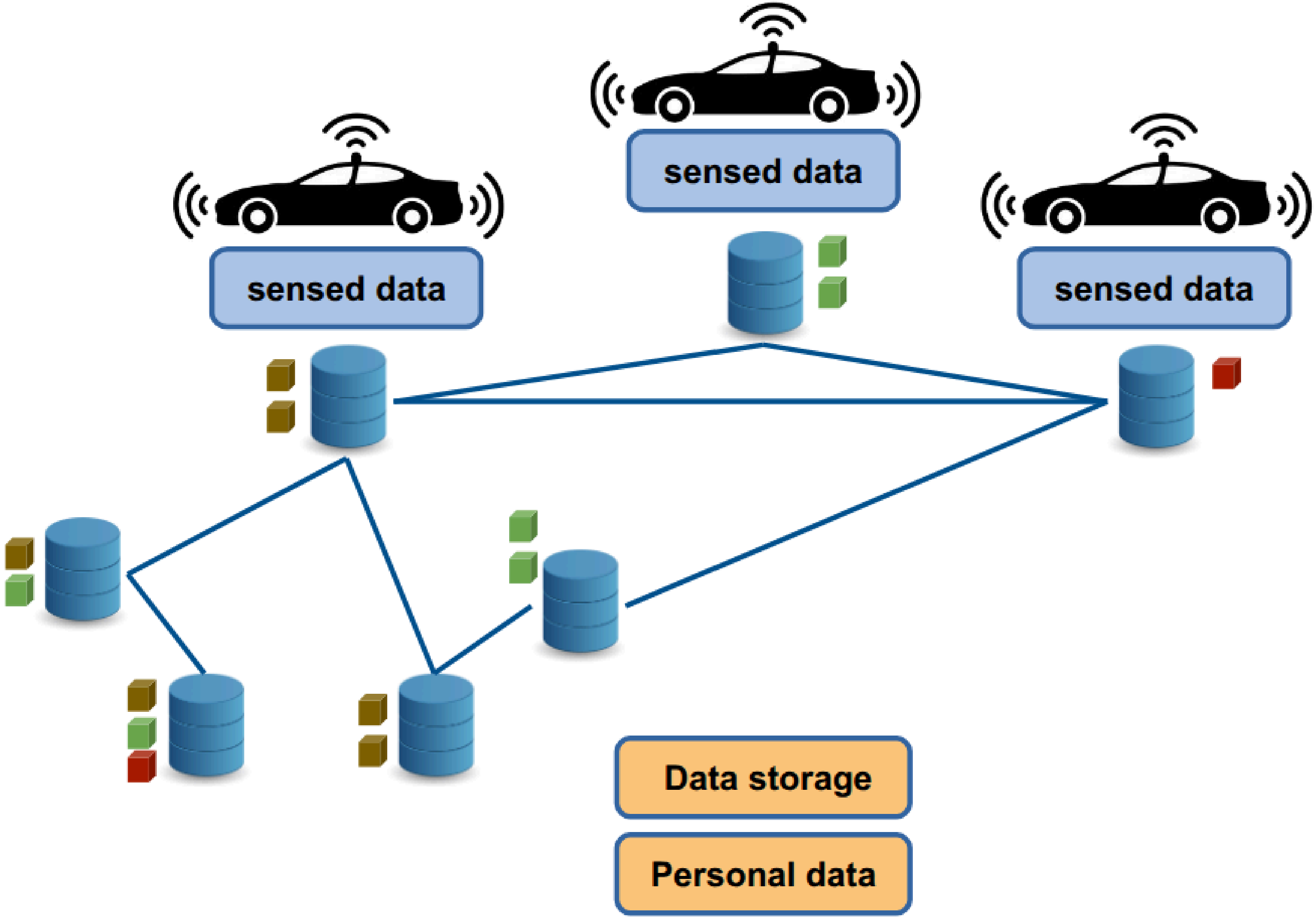
Pros:
- rather simple
- data integrity
- traceability

Data storage

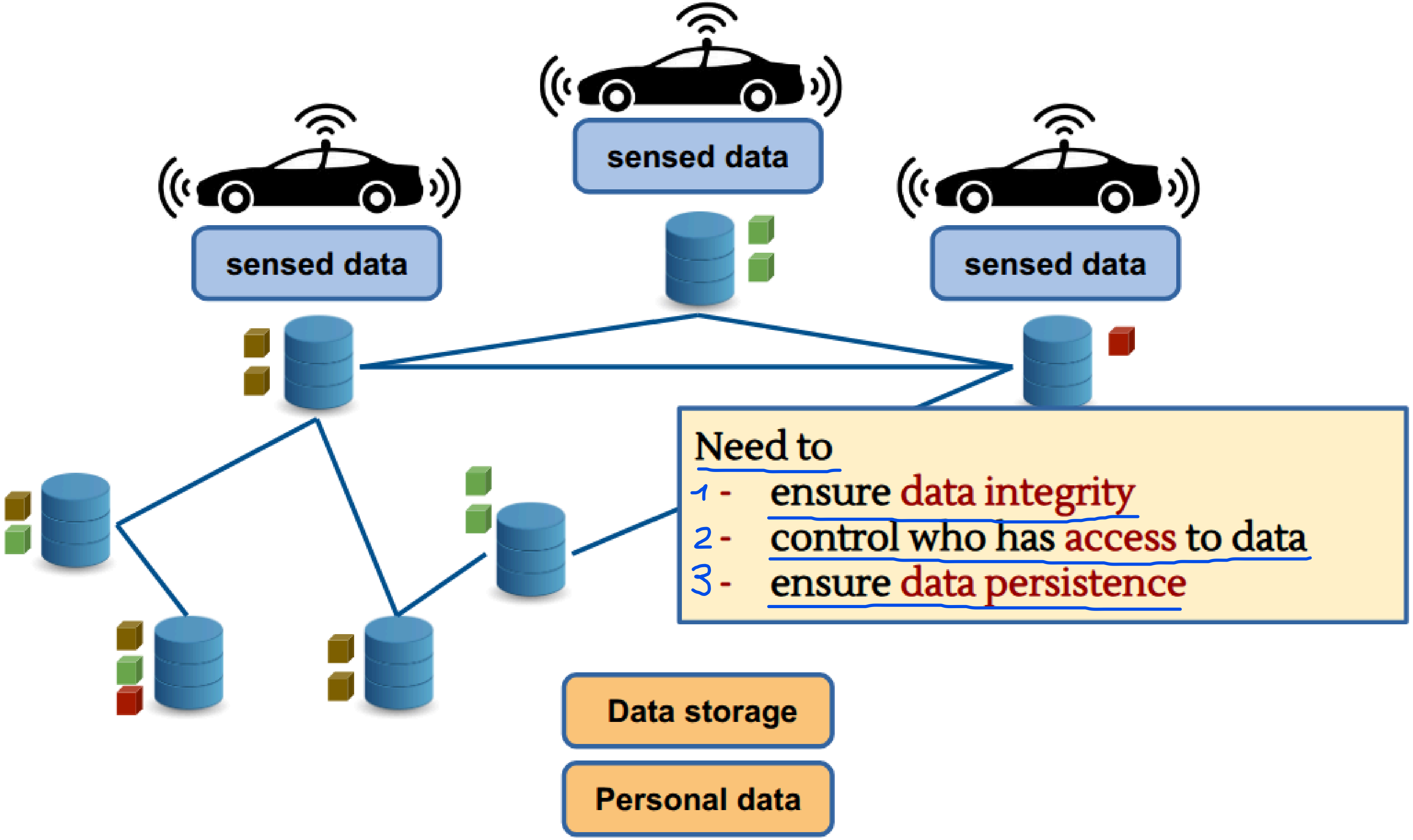
Personal data

Cons:
- ok with small sized data, only
- no right to be forgotten/rectified
- latencies

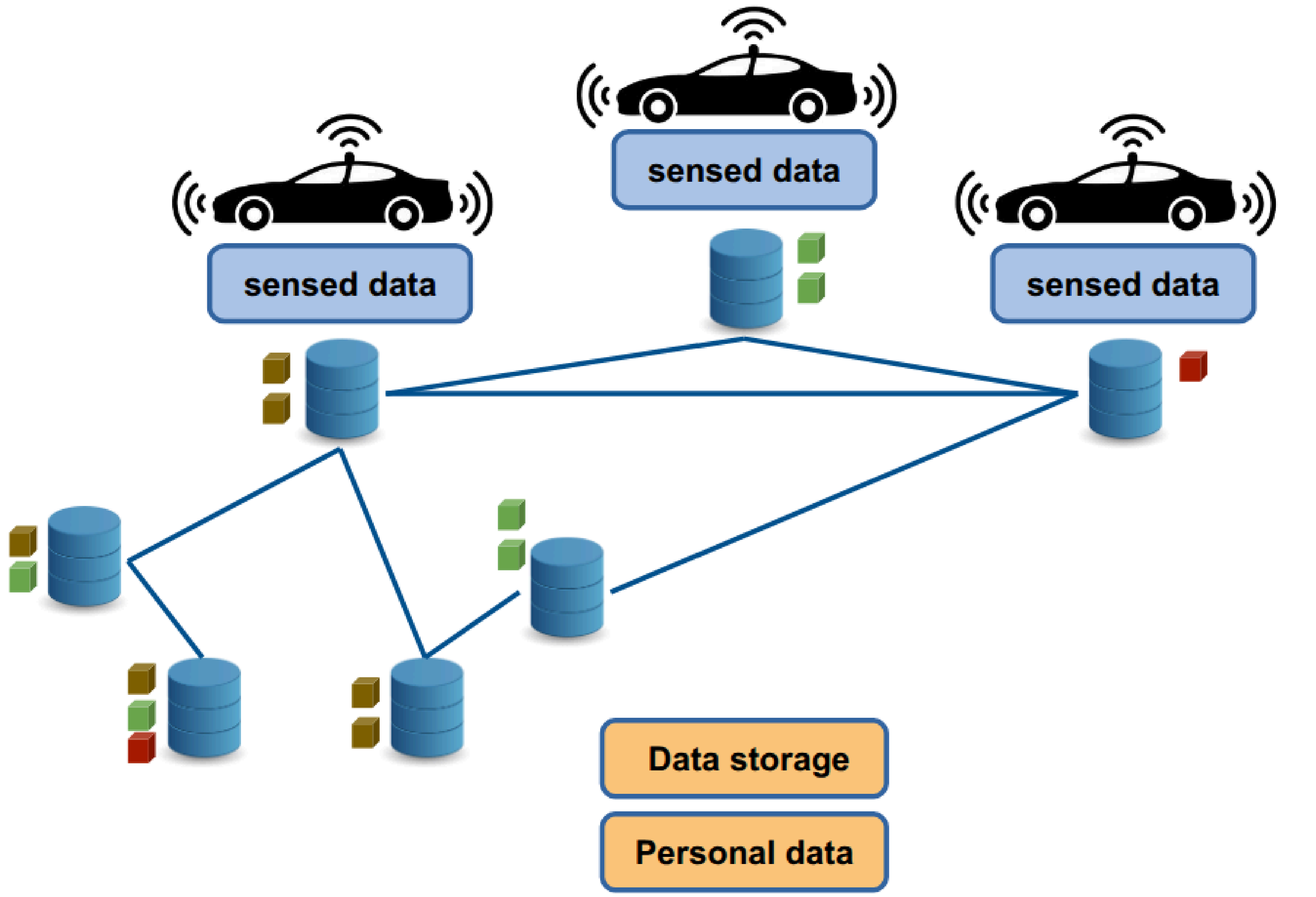
Opt4: use decentralized file systems (DFS)



Opt4: use decentralized file systems



Data Integrity



1

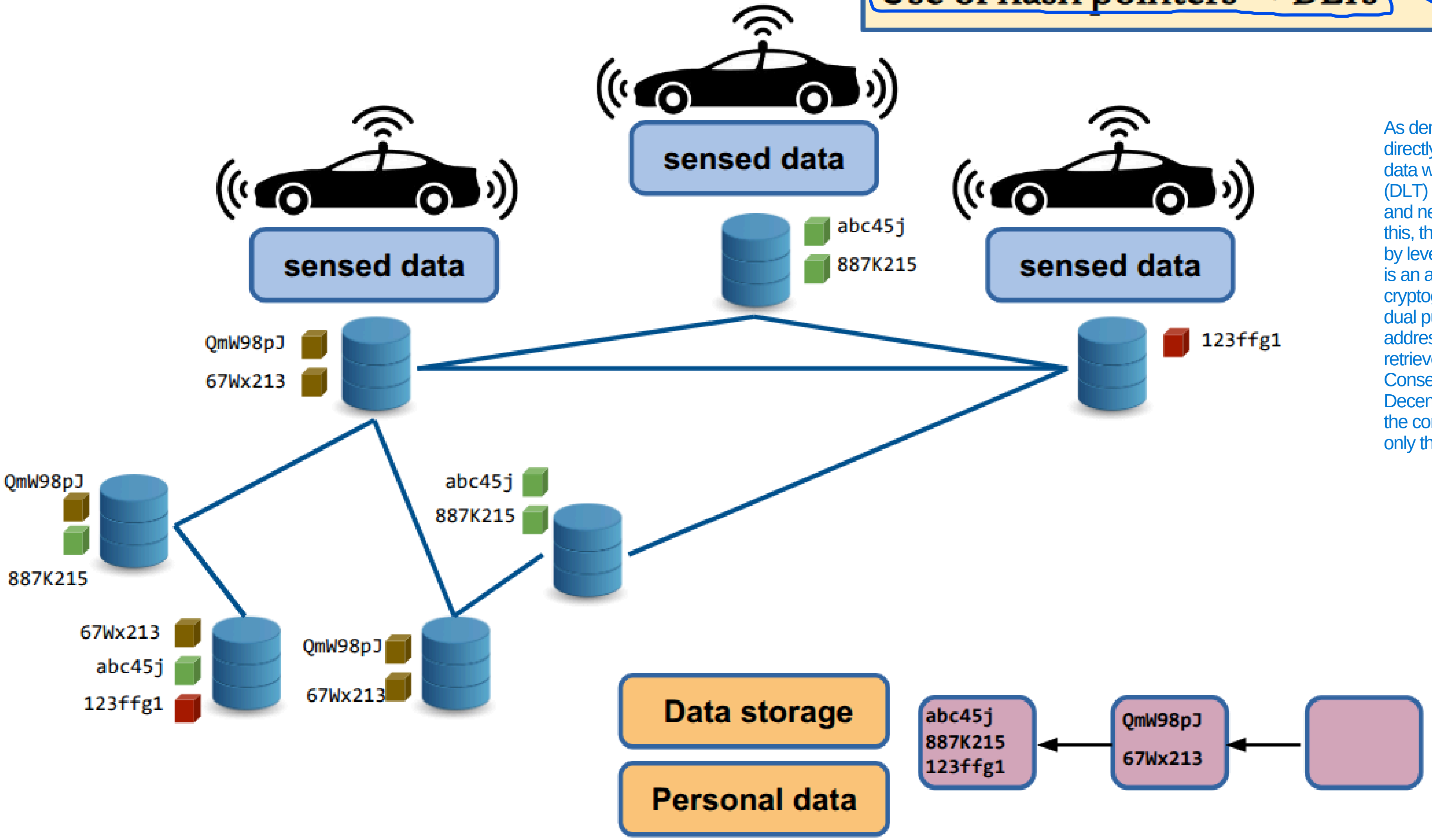
Data Integrity

In decentralized file systems, a file is not located based on its geographic location or the server, but based on its content. The system takes the file's content, calculates its cryptographic hash, and uses that unique string as the address to retrieve it. Consequently, the file's address is the file's digital fingerprint: if you search for that code, the DFS network will find any node that possesses an exact copy of that content.

Content based addressing
(instead of location based)

Use of hash pointers → DLTs

As demonstrated in the analysis of Opt3, directly storing massive volumes of mobility data within a Distributed Ledger Technology (DLT) is unfeasible due to prohibitive costs and network congestion risks. To mitigate this, the architecture splits storage off-chain by leveraging hash pointers. A hash pointer is an address identifier generated from the cryptographic hash of the file itself. It serves a dual purpose: it enables content-based addressing by specifying exactly what data to retrieve, and it guarantees data integrity. Consequently, the user uploads data to the Decentralized File System (DFS), retrieves the corresponding hash pointer, and anchors only this lightweight identifier onto the DLT.



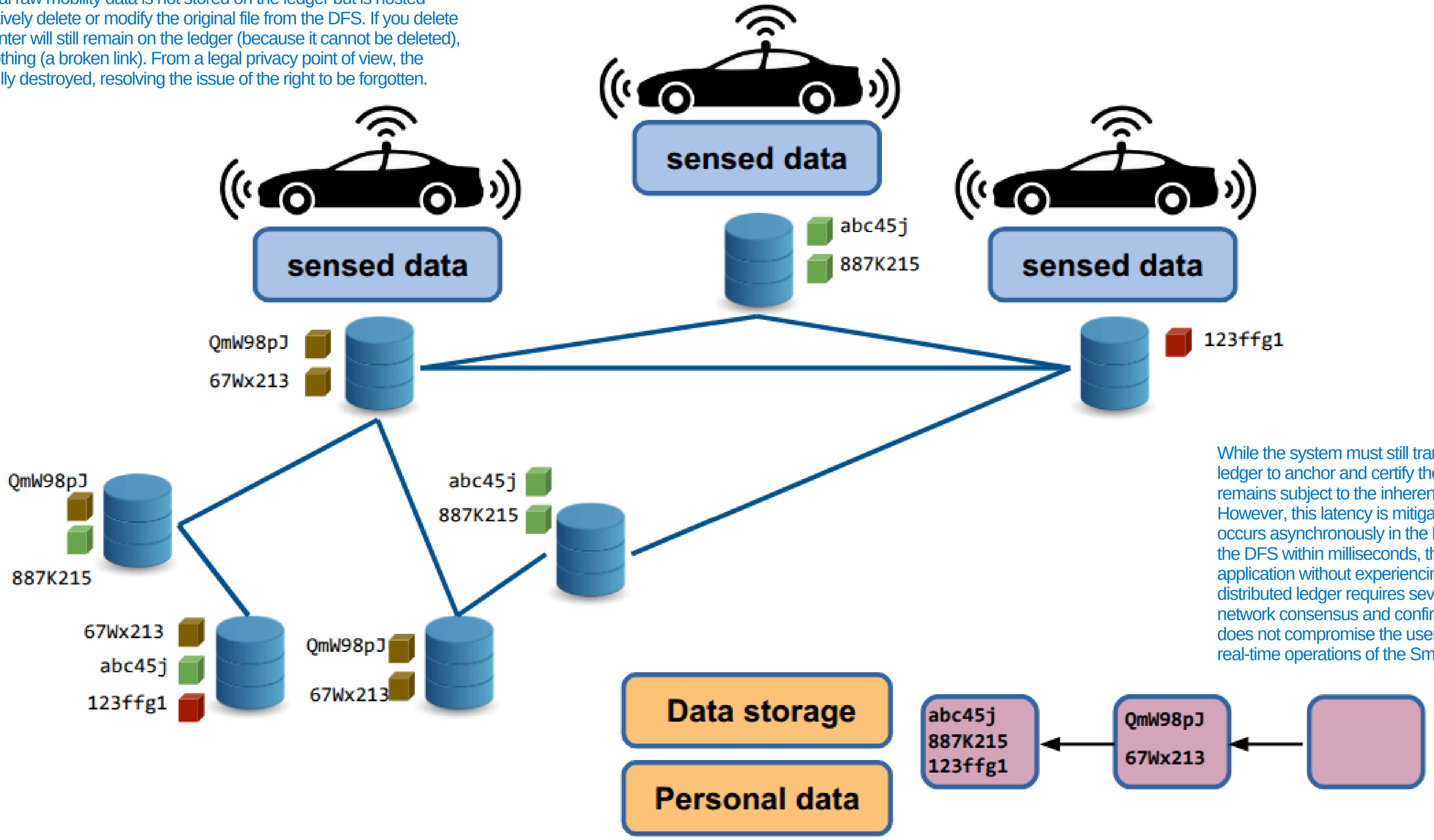
1

Data Integrity

- Possible to remove/modify data
- Fast data upload to DFS
- Latencies to upload hashes less problematic

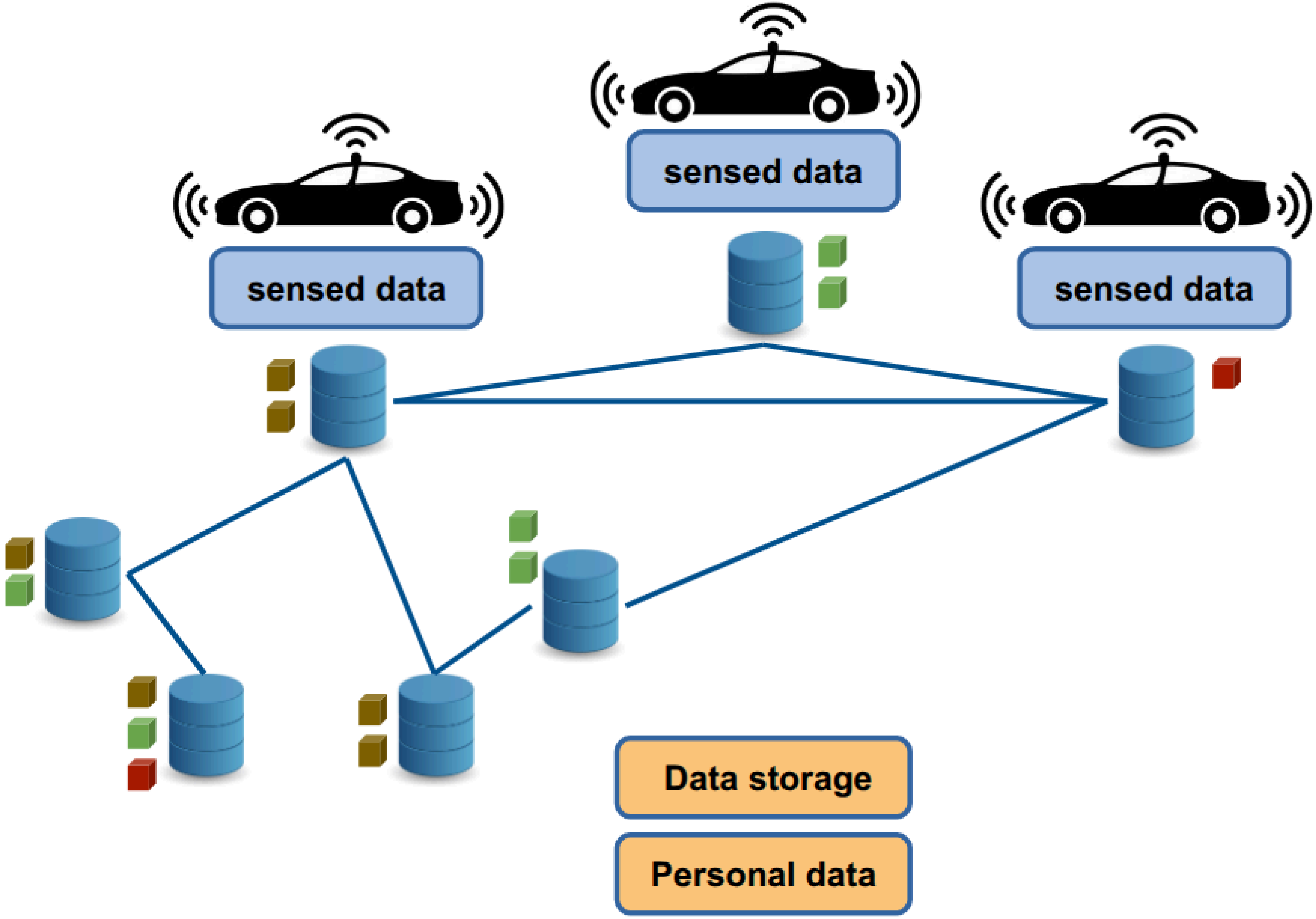
A DFS does not require complex and slow consensus algorithms to accept a file.

Since, in this architecture, the actual raw mobility data is not stored on the ledger but is hosted within the DFS, the user can effectively delete or modify the original file from the DFS. If you delete the file from the DFS, the hash pointer will still remain on the ledger (because it cannot be deleted), but that pointer will now point to nothing (a broken link). From a legal privacy point of view, the personal data has been successfully destroyed, resolving the issue of the right to be forgotten.



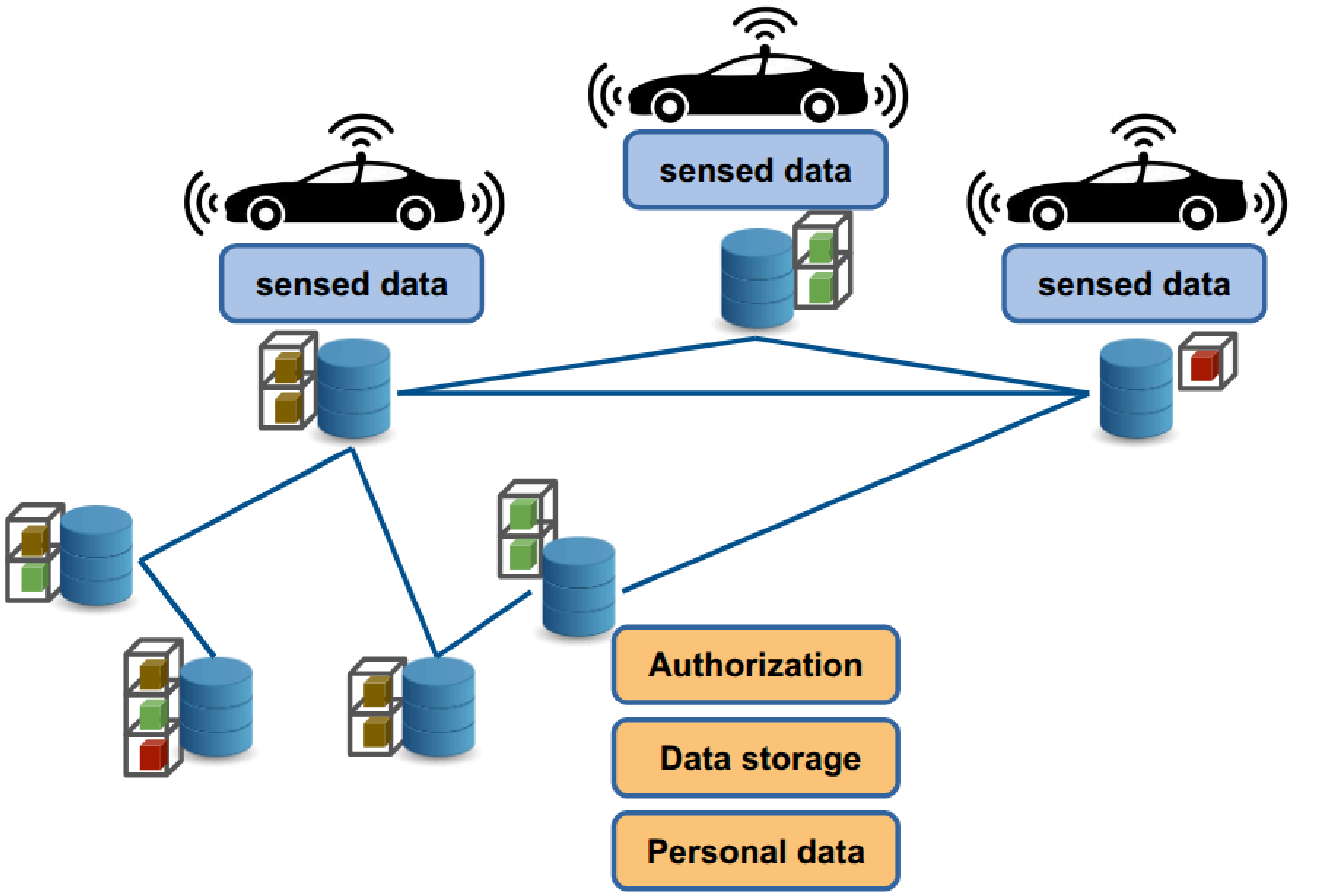
While the system must still transmit the cryptographic hash to the ledger to anchor and certify the file after its rapid upload to the DFS, it remains subject to the inherent latencies of a decentralized network. However, this latency is mitigated because the anchoring process occurs asynchronously in the background. Since the data is saved to the DFS within milliseconds, the user can seamlessly interact with the application without experiencing performance degradation. Even if the distributed ledger requires several seconds or minutes to achieve network consensus and confirm the transaction, this background delay does not compromise the user experience or hinder the safety-critical, real-time operations of the Smart Transportation System.

Access Control?

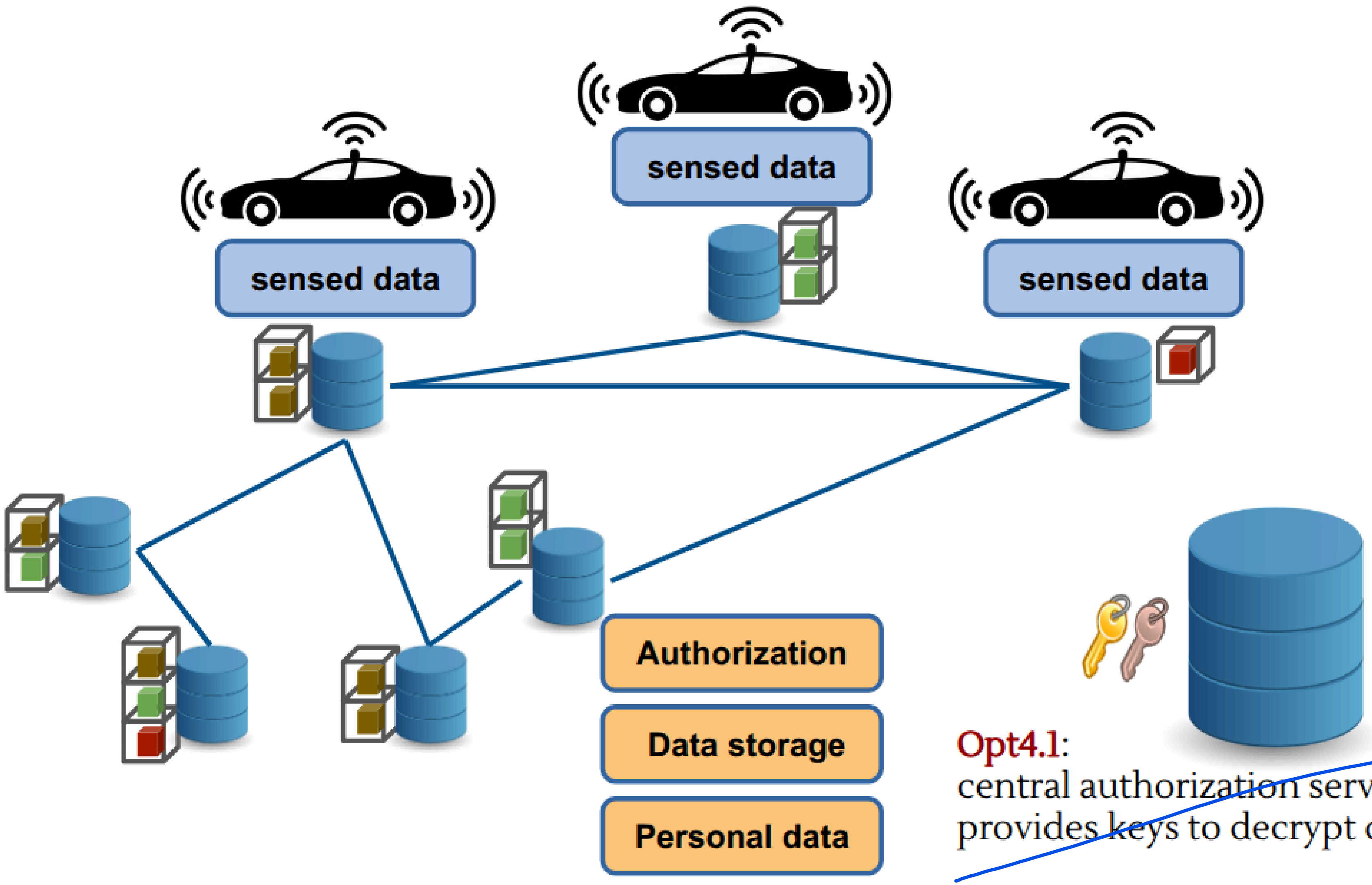


2

Access Control: DFS + Encryption

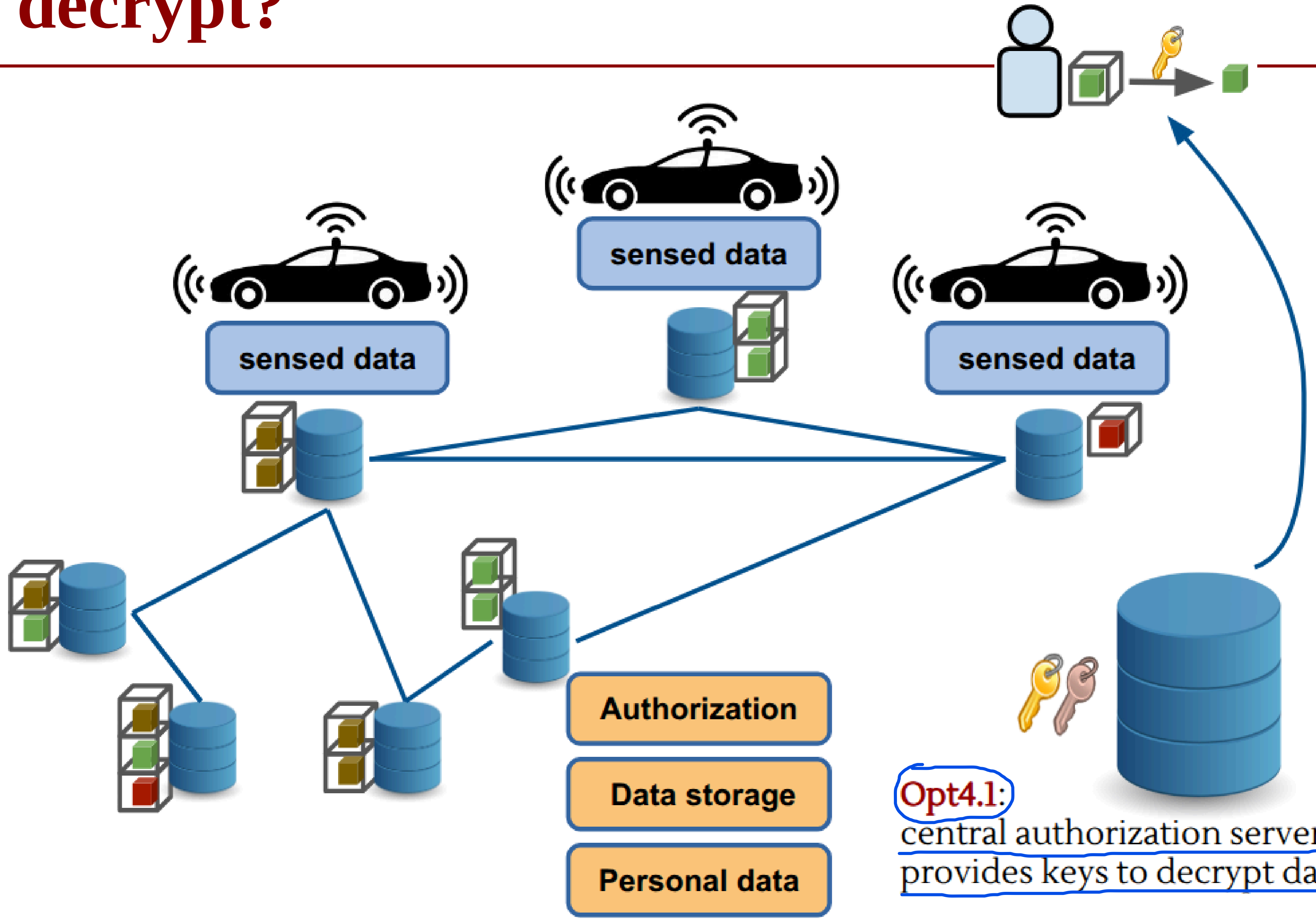


How to decrypt?



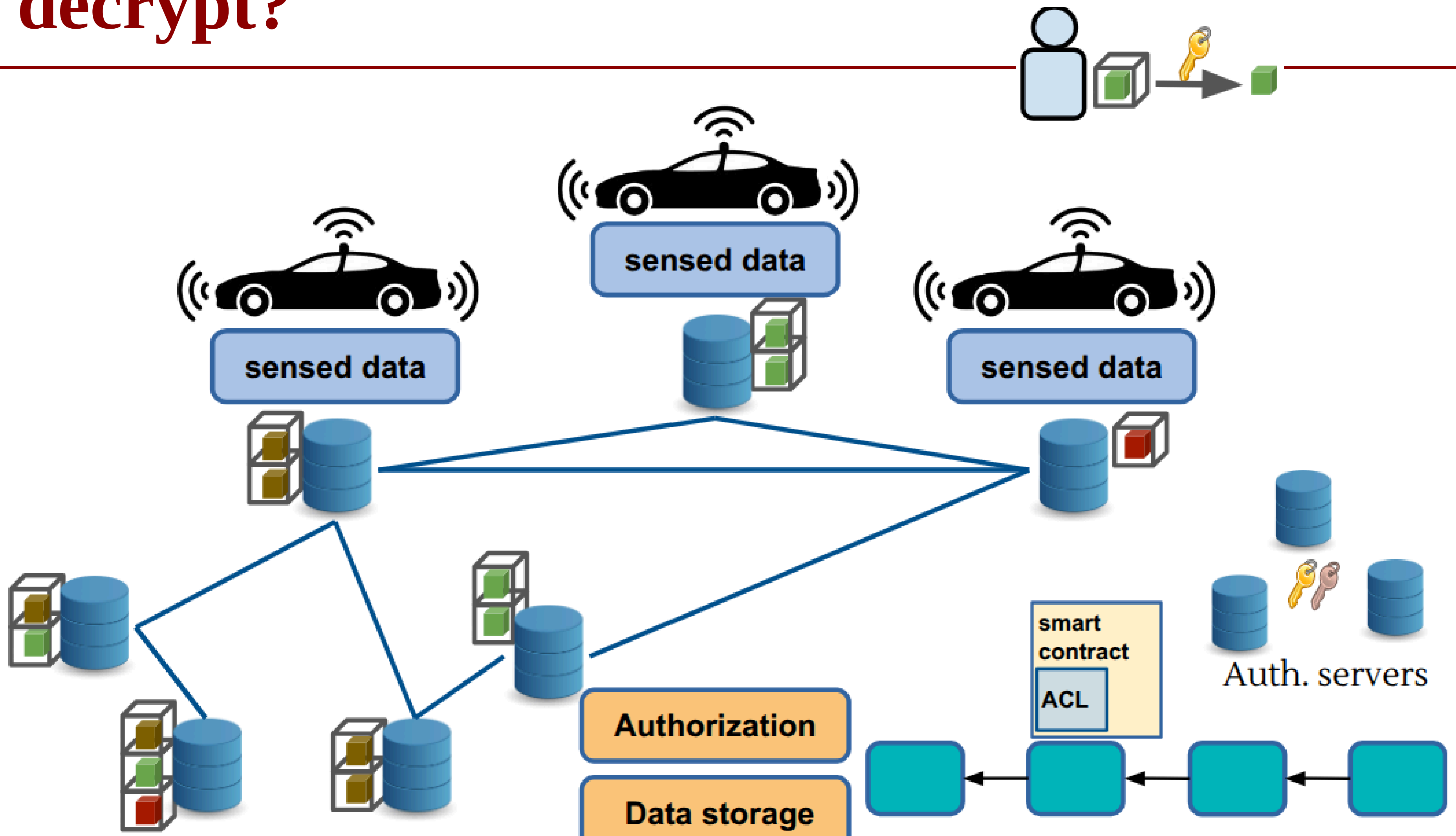
Opt4.1:
central authorization server that provides keys to decrypt data

How to decrypt?



Opt4.1:
central authorization server that provides keys to decrypt data

How to decrypt?



Opt4.2:
security-by-contract through a (private) ledger

How to decrypt?

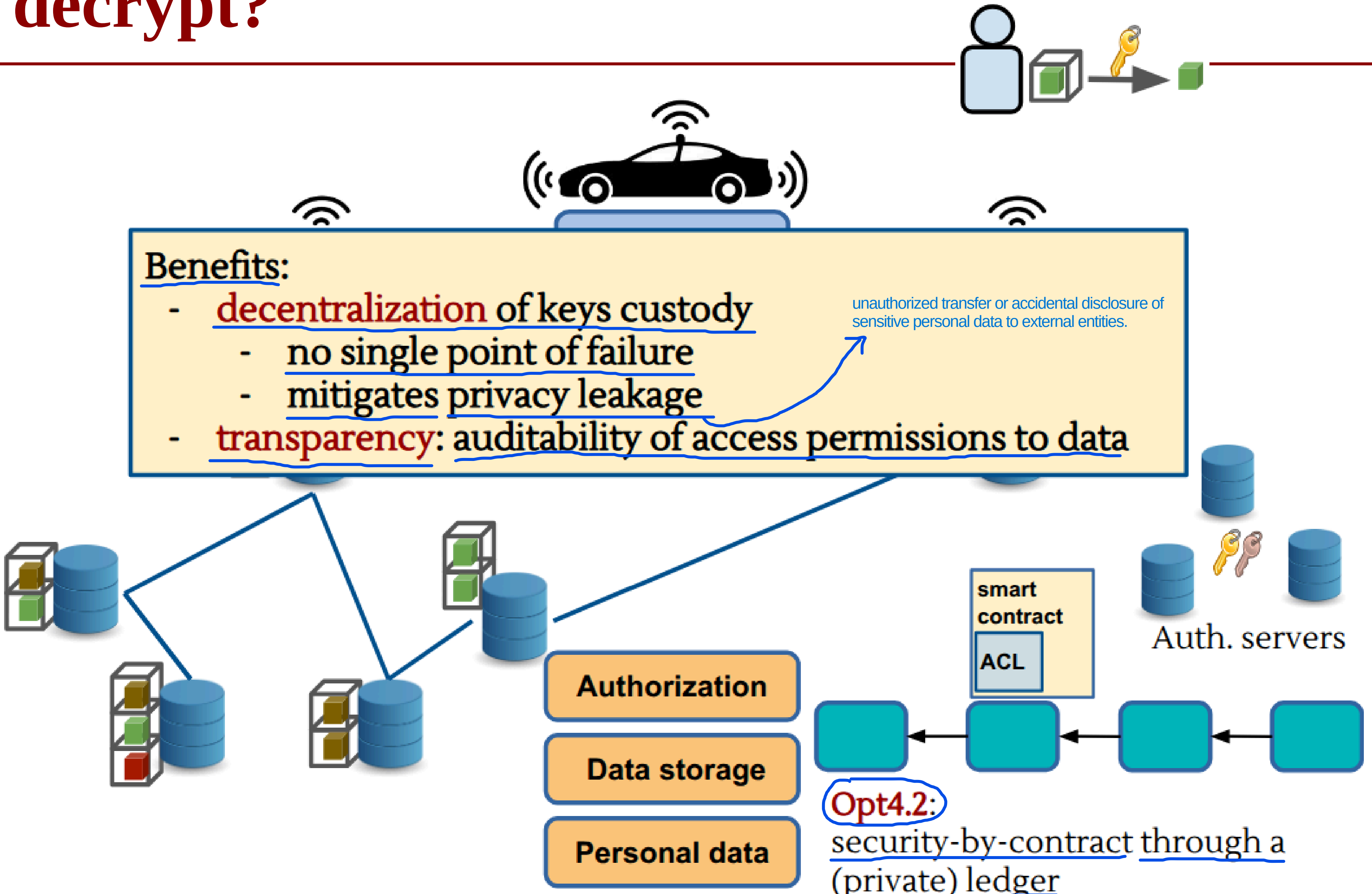
Instead of entrusting the decision of who can view the data to a traditional central server (which represents a single point of failure), security rules are written as code within a smart contract that resides on a private ledger (a private ledger managed by trusted entities).

The Workflow
1. The Request: An application wants to access a sensitive mobility file and makes a request via a network authentication node or server.
2. Contract Verification: The server does not decide on its own, but queries the Smart Contract on the Private Ledger. The Smart Contract evaluates the request automatically and incorruptibly, verifying whether the requester complies with the established digital agreements.
3. Key Release: If the requirements are met, the Smart Contract authorizes the release of the cryptographic key needed to decrypt the file.
4. Download and Decryption: The application downloads the encrypted file from the DFS using its Content Address (the Hash) and, thanks to the key obtained via the Smart Contract, can finally decrypt it and read it in plain text.

Why is it used?
- Confidentiality: No unauthorized user can decrypt the raw data.
- No Single Point of Trust: There is no need to trust a single system administrator; the Smart Contract's mathematical code enforces the rules transparently.
- Auditability: Every time access to data is requested, the action invokes the Smart Contract and leaves an immutable transaction on the Ledger. In the event of abuse, there is a perfect historical record of who accessed what and why.

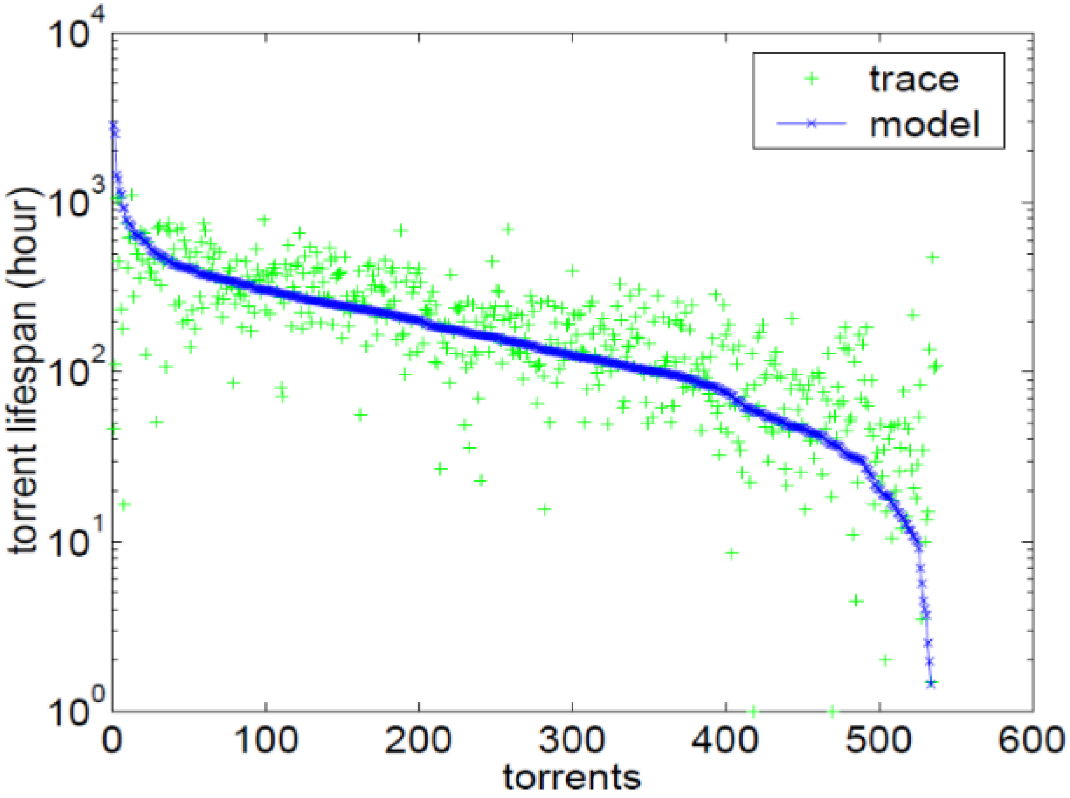
Benefits:
- decentralization of keys custody
  - no single point of failure
  - mitigates privacy leakage
- transparency: auditability of access permissions to data

unauthorized transfer or accidental disclosure of sensitive personal data to external entities.



Opt4.2: security-by-contract through a (private) ledger

Data persistence: we’d like to avoid this

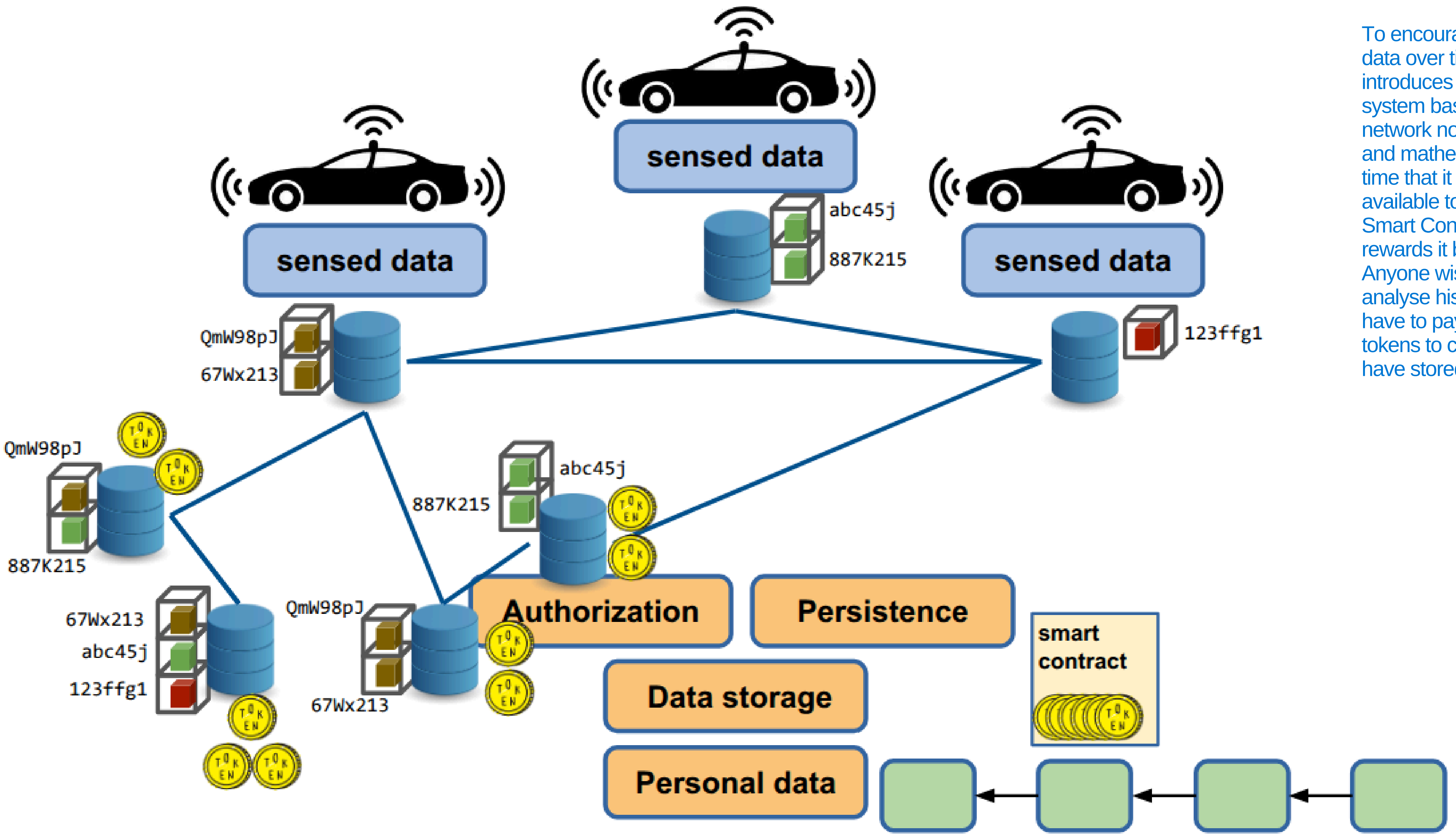


Guo, Lei & Chen, Songqing & Xiao, Zhen & Tan, Enhua & Ding, Xiaoning & Zhang, Xiaodong. (2007). A performance study of BitTorrent-like peer-to-peer systems. Selected Areas in Communications, IEEE Journal on. 25. 155 - 169. 10.1109/JSAC.2007.070116.

3

Data persistence

Incentives to cooperate:
blockchain based tokens
E.g. Filecoin, Sia, Storj, Swarm (??)



To encourage nodes to maintain data over time, the system introduces an economic incentive system based on tokens: if a network node agrees to host a file and mathematically proves over time that it keeps that file intact and available to those who request it, the Smart Contract automatically rewards it by sending tokens. Anyone wishing to download and analyse historical mobility data will have to pay a small amount of tokens to compensate those who have stored that data.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Threat Analysis

Integrity Attacks

- Injection of false mobility data
- Hash substitution attempts
- Replay attacks

Mitigation

- Digital signatures at source
- Hash anchoring on DLT
- Verifiable timestamps

Threat Analysis

Confidentiality Attacks

- Re-identification via mobility traces
- Linkability of pseudonymous identifiers
- Key leakage

Even if GPS data is anonymized, travel patterns reveal unique habits. If an attacker notices a trace that departs every morning at 8:00 AM from a specific residence and arrives at 8:30 AM at a specific office, by cross-referencing this data with data from OSINT, they can accurately trace the driver's real identity.

If a user consistently sends data using the same pseudonym (e.g., a fixed ID or a constant public key), an attacker can " connect the dots" (linkability). Even without knowing the user's real identity, the attacker can reconstruct the entire history of all their past movements over the course of months, creating a perfect profile of them.

If a user mishandles their private key or if the Smart Contract's key management system is compromised, an attacker who gains access to the keys can instantly decrypt all historical files stored on the DFS, rendering all defenses useless.

Mitigation

- Strong encryption
- Key rotation
- Minimal use of metadata

Raw mobility data is encrypted directly on the user's device before being transmitted. This way, if a node in the DFS is compromised, the attacker will find only incomprehensible bits.

To break linkability, the system never uses the same key or identifier indefinitely: they are changed continuously and automatically. If an attacker manages to compromise a key, they will only be able to decrypt a very small fragment of a route lasting a few minutes, without being able to link it to the same user's past or future routes.

The principle of data minimization is applied: when the file is prepared and the hash is sent to the Ledger, all ancillary data not strictly necessary for calculating traffic is deleted. The less metadata there is, the harder it is for an attacker to carry out correlation and re-identification attacks.

Threat Analysis

Availability Attacks

- Denial-of-Service against DFS nodes
- Ledger congestion
- Incentive collapse

An attacker floods the DFS nodes, that host mobility files, with network requests. If these nodes become overloaded, they can no longer distribute files to users or the provider, rendering the entire transport monitoring service unusable.

Every time a vehicle generates a file, it must send the hash to the Ledger to anchor it. During peak hours, millions of cars send transactions simultaneously. If the Ledger becomes congested, the wait times for transaction confirmation increase dramatically (high latency). So, Data is not recorded in a timely manner, making it impossible to monitor traffic in real time.

This is linked to Data Persistence: if the economic value of tokens collapses or if the software reward mechanism fails, the nodes in the P2P network no longer have any economic incentive to waste space and bandwidth to store files. The nodes shut down their servers, and historical data disappears.

Mitigation

- Replication policies
- Multi-node anchoring
- Layered fallback mechanisms

To counter DoS attacks on DFS nodes, the system enforces redundancy rules. Whenever a user uploads a file, it is not saved on a single node, but is automatically copied and replicated across dozens of geographically dispersed nodes. If an attacker takes a node offline, the network will instantly retrieve the same file from any of the other active mirror nodes.

Instead of requiring all vehicles to send their hashes to a single ledger node, the in-vehicle application is designed to interface with a dynamic pool of validator nodes on the ledger.

If the main decentralized network experiences total congestion or an incentive collapse, the system seamlessly scales to backup layers. For example, if the Ledger is temporarily blocked, the user's device locally buffers the hashes in a queue (local buffering) and then sends them in bulk as soon as the network clears, preventing the immediate loss of mobility flows.

Lessons Learned (Crowdsensing)

What really matters

- Data is **distributed**, but risks are **centralized through aggregation**
- **Integrity, privacy, and availability** must be designed together
- Architecture choices directly impact **trust assumptions**

Every architectural choice shifts the trust boundary: if you choose a traditional centralized solution, your assumption is: "I blindly trust the central server." If you choose Opt4, your assumption changes: "I don't trust anyone; I only trust the mathematics of the Ledger and the Smart Contract." The architecture defines who (or what) you must trust.

What we need to study

- Secure data sharing
- Distributed systems
- Privacy-preserving techniques (anonymization, aggregation)

Use Case #2: Security Analysis of a Clinical ML System

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Scenario

Federated Learning:
- Data remains local: every hospital keep their data on their own secure servers.
- Distributed training: a central cloud orchestrator sends a base version of the diagnostic model to each hospital. Each hospital trains the model locally using only its own data. Hospitals do not send patient data to the cloud, but only the mathematical weights. The cloud orchestrator aggregates these parameters to create an improved global model, which is then redistributed to everyone (the weights).
- Clinicians then query this global model via a Web API: they send the current data of a patient in the ICU, and the API instantly returns the percentage risk of readmission.

A diagnostic model is trained on data collected from **multiple hospitals**

The model is deployed as a **cloud-orchestrated federated learning system** and accessed through a web API by clinicians

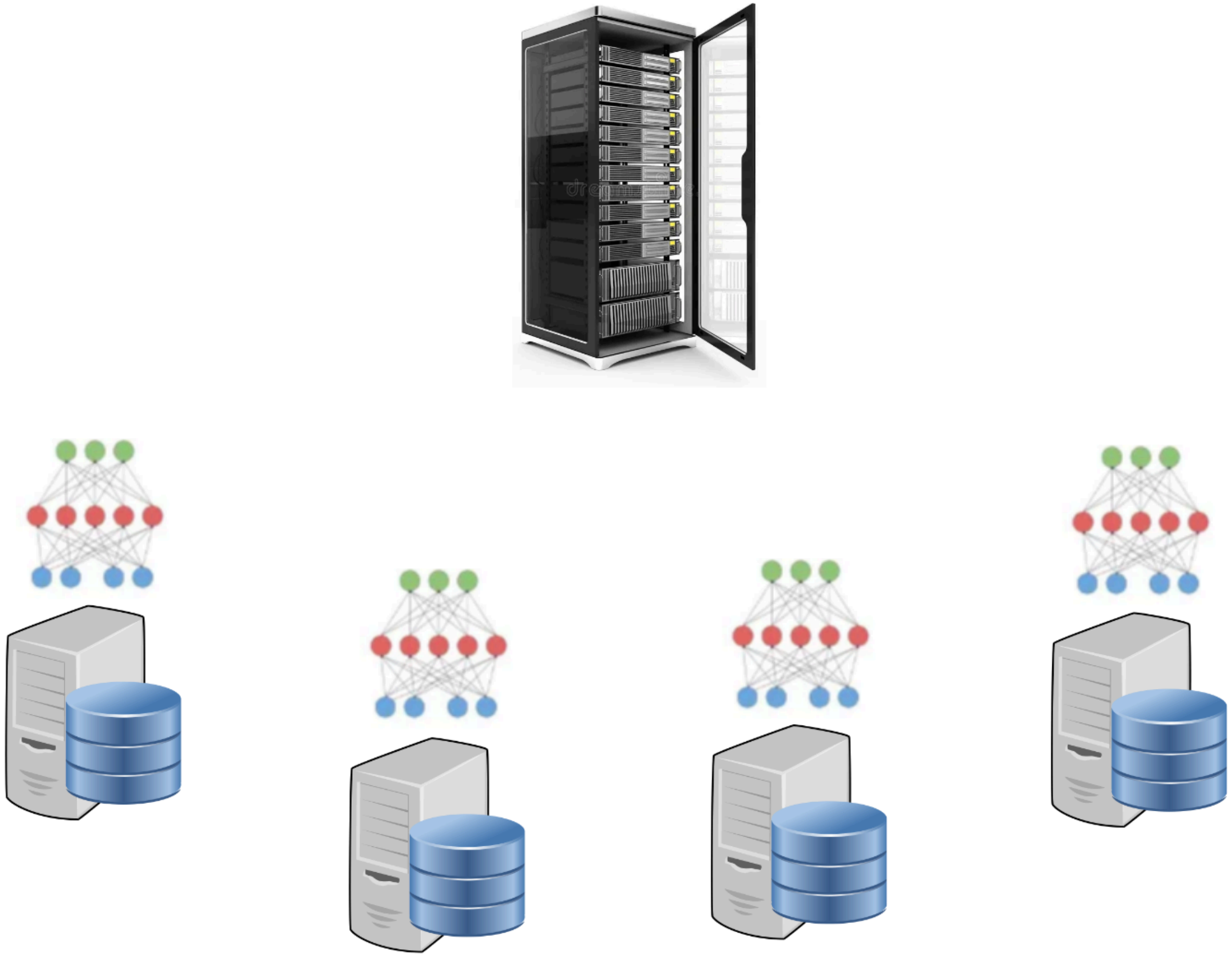
Function: Predict ICU (Intensive Care Unit) readmission risk → refers to using machine learning models to calculate the probability that a patient, once discharged from the ICU to a standard hospital unit or home, will deteriorate and require readmission to the ICU within a specific timeframe (typically 48 to 72 hours, or up to 30 days).

The system must satisfy:

- Regulatory constraints (GDPR compliance) →
  - Data Minimization & Sovereignty: Raw patient data never leaves the physical and legal boundaries of the originating hospital.
  - Privacy-Preserving Techniques: Since a sophisticated attacker could theoretically “reconstruct” certain patient data by analyzing the mathematical weights sent to the cloud, techniques such as Differential Privacy (adding controlled statistical noise to the gradients) are used.
- Clinical reliability requirements →
  - Domain Bias Mitigation: Hospitals have different patient populations, equipment, and clinical guidelines. The federated model must be validated to prevent metrics from a large hospital from overshadowing the specific characteristics of a smaller clinic.
  - Explainable AI (XAI): The API should not only provide a percentage (e.g., “Re-admission risk: 85%”), but should also explain which clinical parameters (blood pressure, ejection fraction, blood parameters) drove the prediction, allowing the clinicals to verify the clinical rationale behind the alert.
- Security and resilience against adversarial manipulation

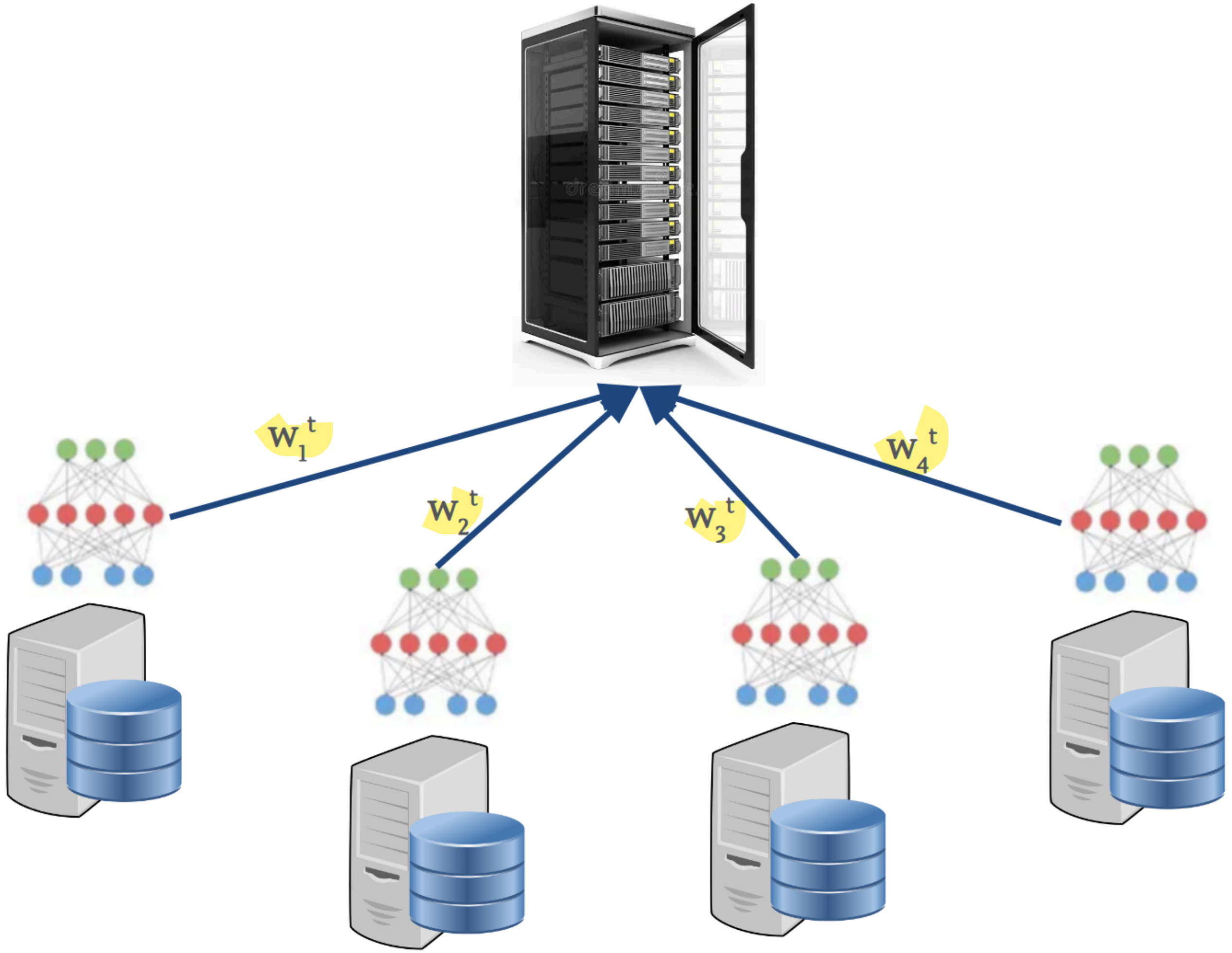
System Architecture

Training Paradigm: **Federated Learning** of the model **(FL)** across N hospitals



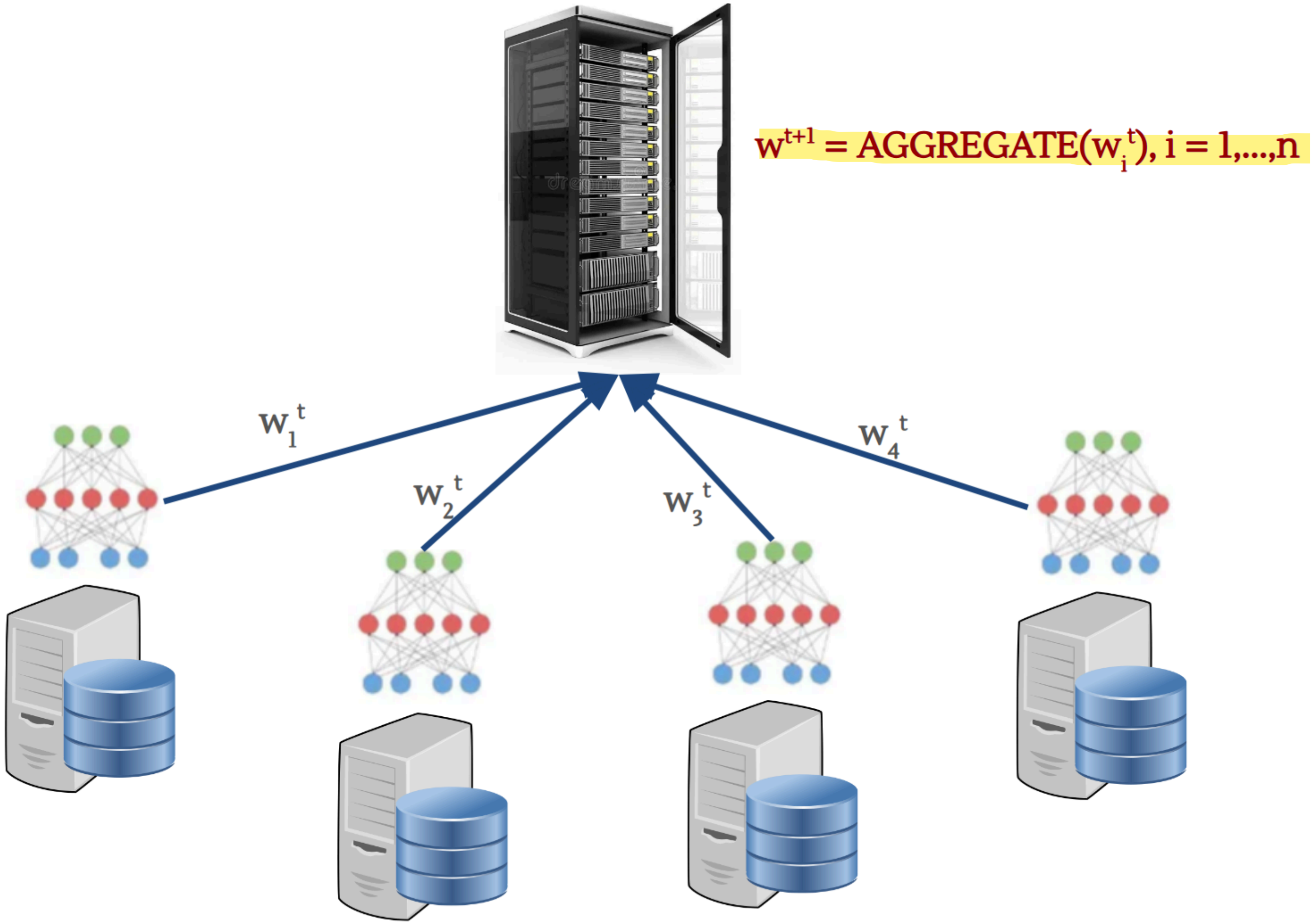
System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



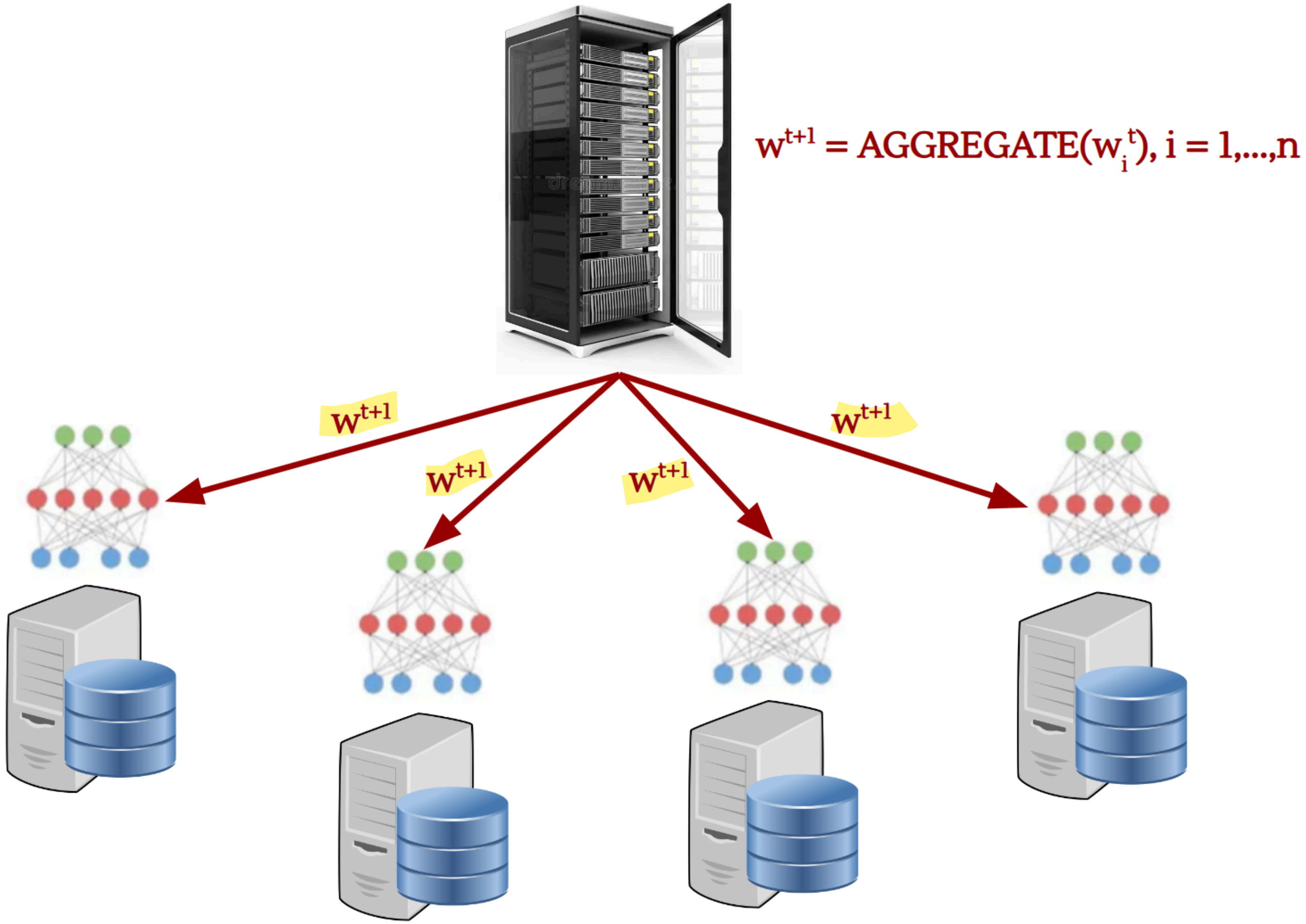
System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



System Architecture

Data Characteristics

- Patient records remain local to each hospital
- Model updates (gradients or weights) are transmitted to a central aggregator
- During **training**, labels are produced by external annotation contractors that have access to local data

Deployment

- Cloud-based infrastructure
- API-based access for clinical users

Threat Modeling Framework

We classify threats according to three primary security objectives:

- **Integrity**: correctness of model parameters and outputs
- **Privacy**: protection of patient information
- **Availability**: continuity and reliability of clinical service

Integrity Threat 1: Data Poisoning

Attack Surface

Training labels provided by external contractors

Integrity Threat 1: Data Poisoning

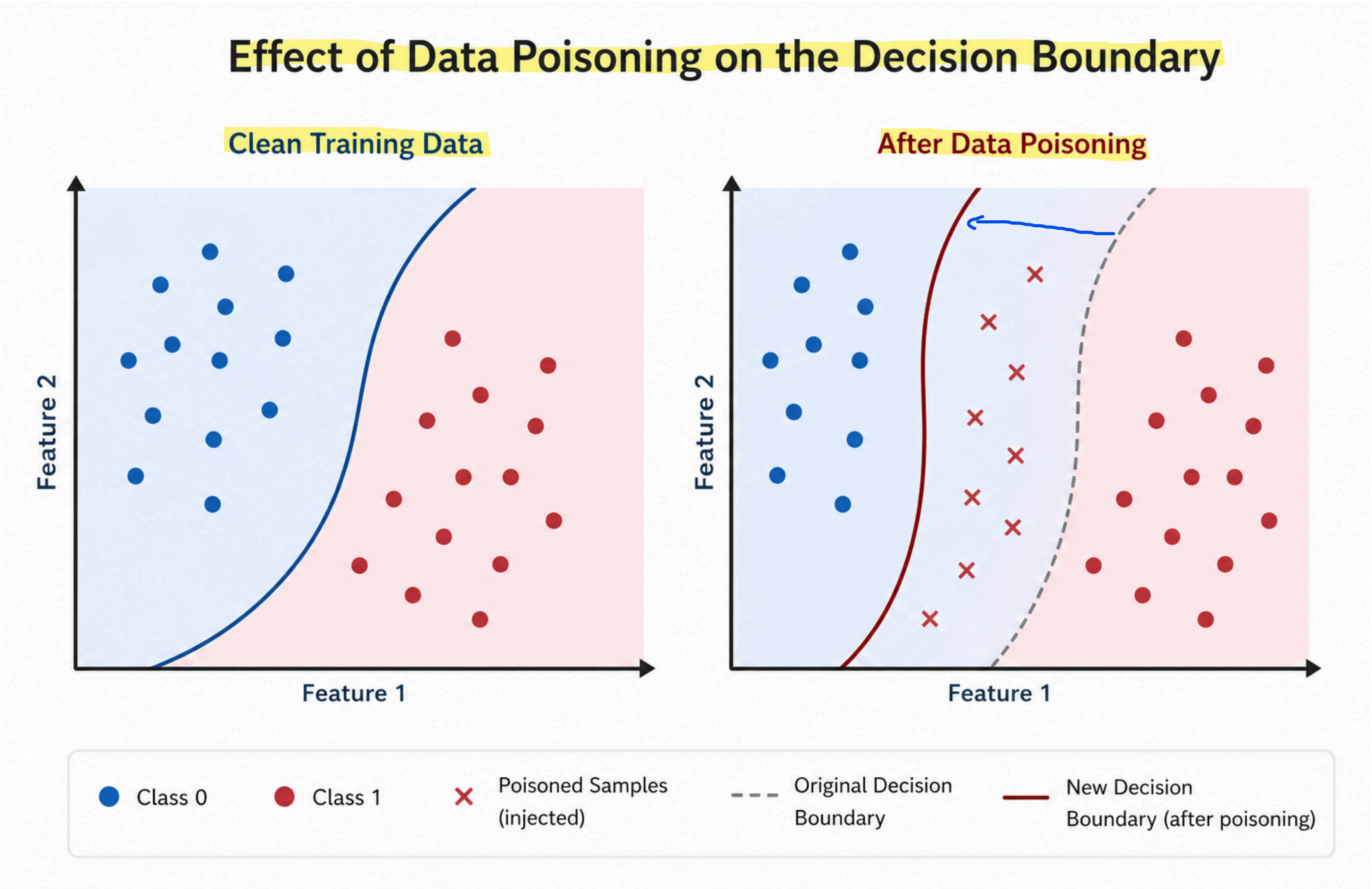
Attack Surface

Training labels provided by external contractors

Threat Description

- Incorrect or maliciously altered labels degrade model correctness
- Systematic bias may be introduced in specific sub-populations

Integrity Threat 1: Data Poisoning



Integrity Threat 1: Data Poisoning

Measurable Mitigation

Valutazione dell'affidabilità tra valutatori

Inter-Rater Reliability (IRR) Scoring

- Each record labeled by ≥ 3 annotators
- Discard or re-label samples below a defined agreement threshold

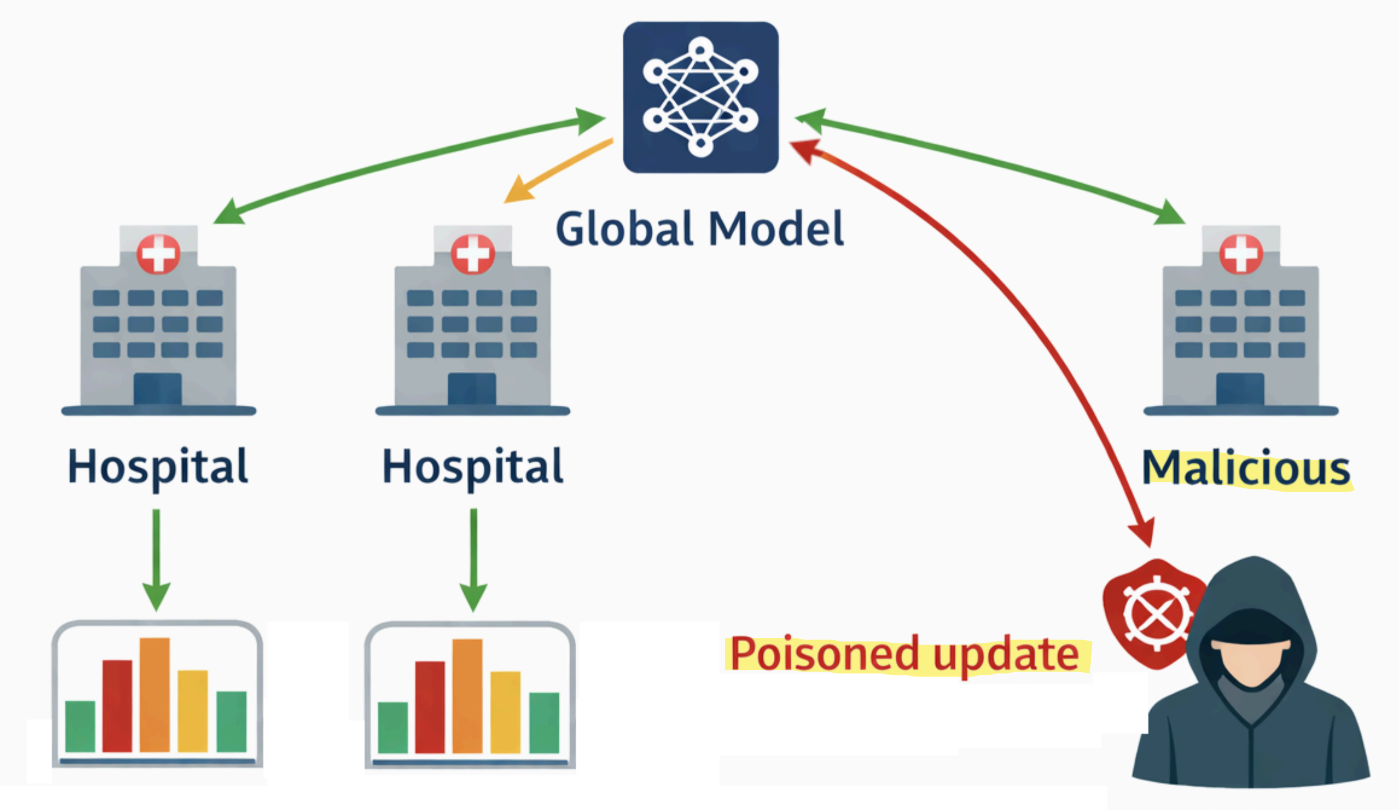
Trade-off

- Increased annotation cost (approximately linear in redundancy factor)
- Increased dataset preparation latency
- Possible reduction in effective dataset size

Integrity Threat 2: Poisoned Federated Updates

Attack Surface

Model updates transmitted by hospitals in the FL protocol



Integrity Threat 2: Poisoned Federated Updates

Attack Surface

Model updates transmitted by hospitals in the FL protocol

Threat Description

A compromised participant sends adversarial gradients to bias the global model

Effects may include:

- Targeted backdoor behavior

The attacker secretly teaches the model to behave abnormally only in the presence of a specific trigger (backdoor), while in all other cases the model continues to function normally. Example: The overall model correctly predicts 85% of readmission risks, but the attacker has inserted a hidden rule: if the patient is exactly 64 years old and has a specific hemoglobin value, the model will always predict Low Risk, even if the patient is in critical condition. This allows an attacker to intentionally manipulate the diagnosis of a specific target without being detected by general accuracy checks.

- Systematic degradation of specific predictions

The attack aims to undermine the model's reliability across entire categories of data by drastically reducing its accuracy for specific groups. Example: Poisoned gradients destabilize training related to patients with heart disease. As a result, the model becomes unable to correctly predict readmission to the ICU for heart patients, leading to erroneous medical decisions on a systemic scale for that specific category of patients.

Integrity Threat 2: Poisoned Federated Updates

Measurable Mitigation

Robust Aggregation Rules

In standard federated learning, the classic algorithm (FedAvg) calculates a simple weighted average of all the gradients received. If a hospital sends a poisoned gradient with extremely large or distorted values, the average is heavily distorted. Robust Aggregation Rules replace the classic average with advanced mathematical functions capable of withstanding Byzantine attacks

Examples:

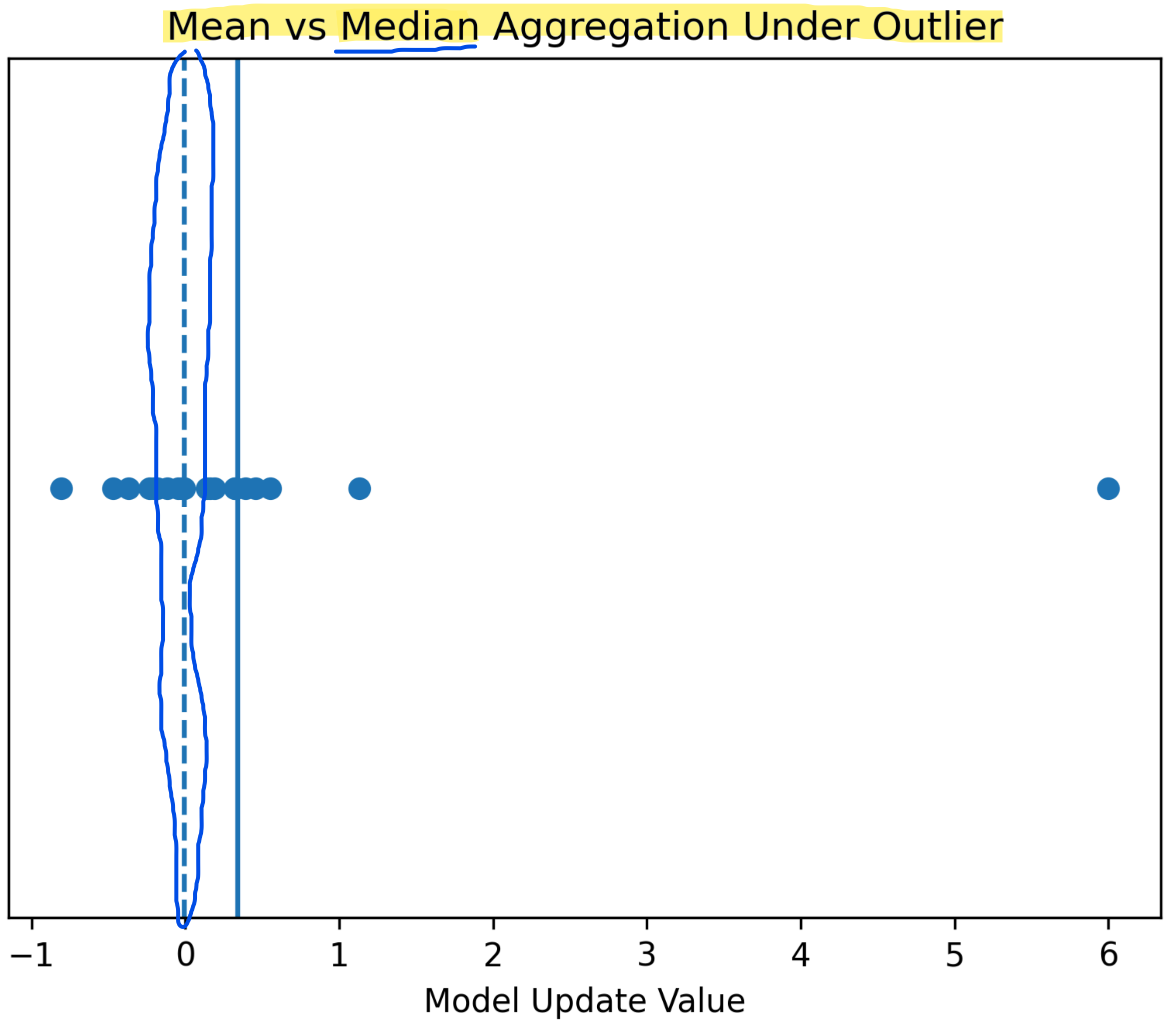
- Krum
- Coordinate-wise median
- Trimmed mean

Metric:

- Bounded influence of outlier updates

This is the fundamental metric for measuring the effectiveness of the mitigation. A system is considered secure if the maximum impact that a malicious node can have on the global model is strictly limited and bounded. By applying Krum, the Median, or the Trimmed Mean, the mathematics ensures that even if an attacker sends gradients with infinitely large or heavily distorted values to create a backdoor, the aggregation algorithm will nullify or reduce their influence to a negligible amount, preserving the integrity and clinical reliability of the final diagnostic model.

Integrity Threat 2: Poisoned Federated Updates



Integrity Threat 2: Poisoned Federated Updates

Trade-off

- Reduced sensitivity to legitimate distributional heterogeneity
- Potential degradation of performance on minority subpopulations
- Increased aggregation complexity

Privacy Threat: Membership Inference

Attack Surface

Public-facing inference API

Threat Description

An adversary queries the model to determine:

1 • if it is possibile to know that ~~Whether~~ a specific patient record was used in training

2 • if it is possibile that ~~Whether~~ sensitive attributes can be reconstructed

Relevant attacks:

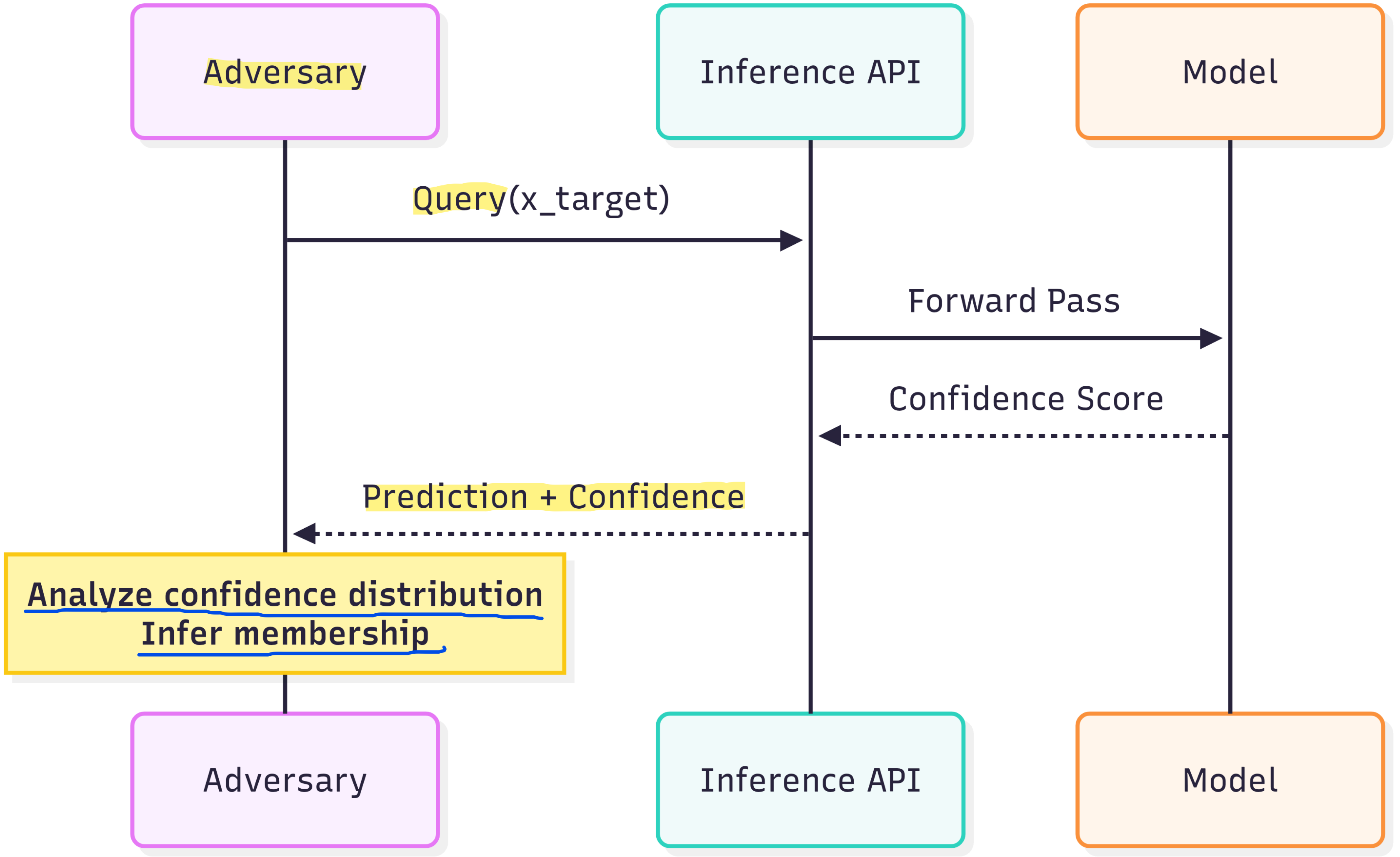
1 • Membership inference

Determining with high statistical certainty whether a specific individual's or patient's record was included in the model's training dataset.

2 • Model inversion

Inferring and reconstructing highly sensitive, hidden attributes of a data subject by analyzing the model's outputs and prediction confidence scores.

Privacy Threat: Membership Inference



Privacy Mitigation: Differential Privacy

Mechanism

The fundamental principle is based on the concept of indistinguishability: imagine you have a dataset D (e.g., the clinical data of patients at a hospital) and a neighboring dataset D', which contains exactly the same data as D, except for a single record. An AI model satisfies Differential Privacy if, when run on D or on D', it produces an output with an almost identical probability distribution. In simple terms, if the result of the medical analysis or the model does not change significantly whether I participate in the study or not, an attacker observing the output will never be able to tell whether my personal data was present in the system or not, neutralizing the risks of re-identification.

- **Differential privacy (DP)** is a *mathematically rigorous privacy definition* where one dataset vs its neighbor should not significantly change the output probability distribution
- DP is commonly described via the (ε, δ)-condition

- Epsilon controls the level of privacy: it represents the maximum multiplier of the probability difference between the two datasets. The smaller epsilon is (closer to zero), the more identical the output across the two datasets will be, ensuring very strong privacy. On the other hand, an epsilon that is too small introduces a lot of noise, reducing the model's clinical accuracy.
- Delta represents the probability that the strict privacy constraint will fail: a sort of margin of error or probability of a small information leak. In secure systems, delta must be extremely small.

Formal Guarantee

For adjacent datasets D, D' differing in one record:

$$P(M(D) \in S) \leq e^{\varepsilon} P(M(D') \in S) + \delta$$

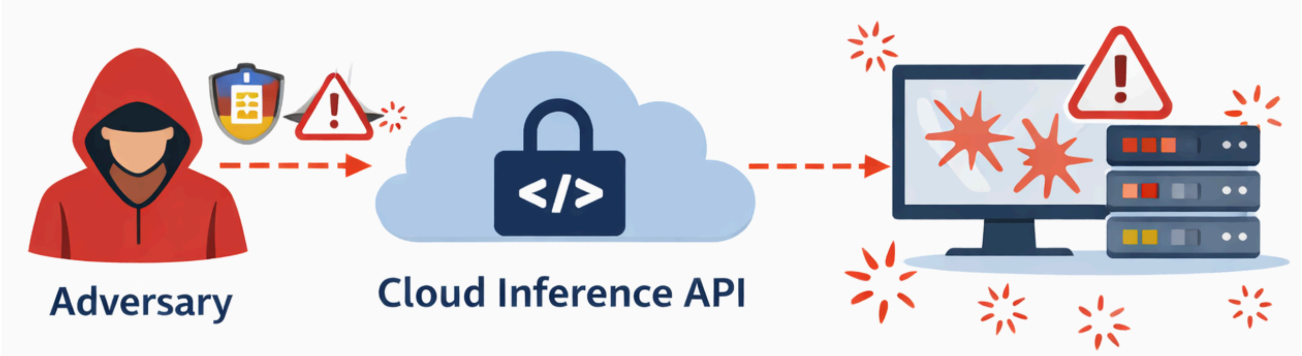
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Availability Threat: Adversarial Denial of Service

Attack Surface

Cloud inference API

- Cloud Inference API (The Infrastructure View): This highlights where the model lives and runs. The model is deployed on a cloud-based server infrastructure rather than running locally.
- Public-facing Inference API (The Accessibility View): This highlights who can reach it and how. "Public-facing" means that the API endpoint is exposed to the network, allowing external clients (like authorized clinicians or potential attackers) to send input data and receive predictions.



Threat Description

Adversarial inputs crafted to:

- Maximize computational load ("sponge samples")
- Trigger numerical instability
- Cause repeated exceptions

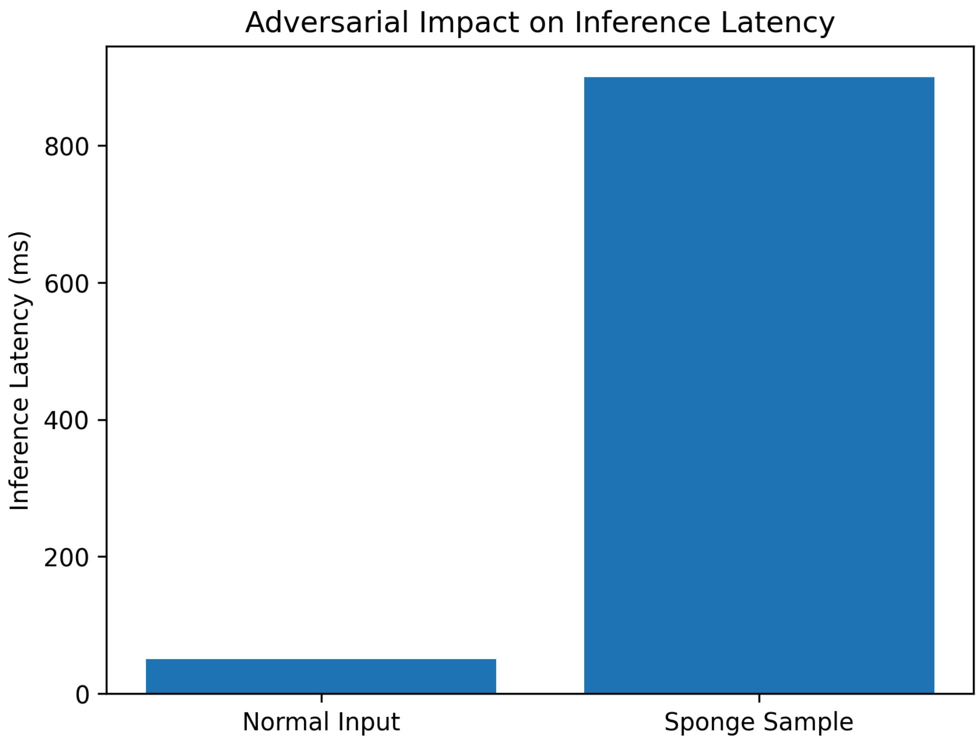
Availability Threat: Adversarial Denial of Service

Effect

- Increased latency
- Degraded clinical workflow refers to a breakdown in standard patient care processes
- Potential service outage

Controls

1. API rate limiting per authenticated user
2. Input validation and anomaly detection
3. Resource caps per inference request



Availability Mitigation

Measurable Metrics

- Maximum latency per request
- Rejection rate of anomalous inputs
- Service uptime percentage

Indicates the maximum time (in milliseconds) it takes for the system to respond to a single medical query. If latency spikes (e.g., from 50ms to 800ms), it means that a DoS attack or a " sponge sample" is congesting the servers.

This is the percentage of requests that the security system blocks at the entry point because they are considered strange, potentially dangerous or manipulated. A sudden spike in this metric signals an ongoing attack that the defence is neutralizing.

This is the classic availability metric (e.g., 99.9%). It measures the fraction of time during which the cloud API is active, functioning, and accessible to clinicals without interruption.

Trade-off

- False rejection of legitimate but atypical clinical cases
- Operational friction in emergency contexts

Lessons Learned (Clinical ML)

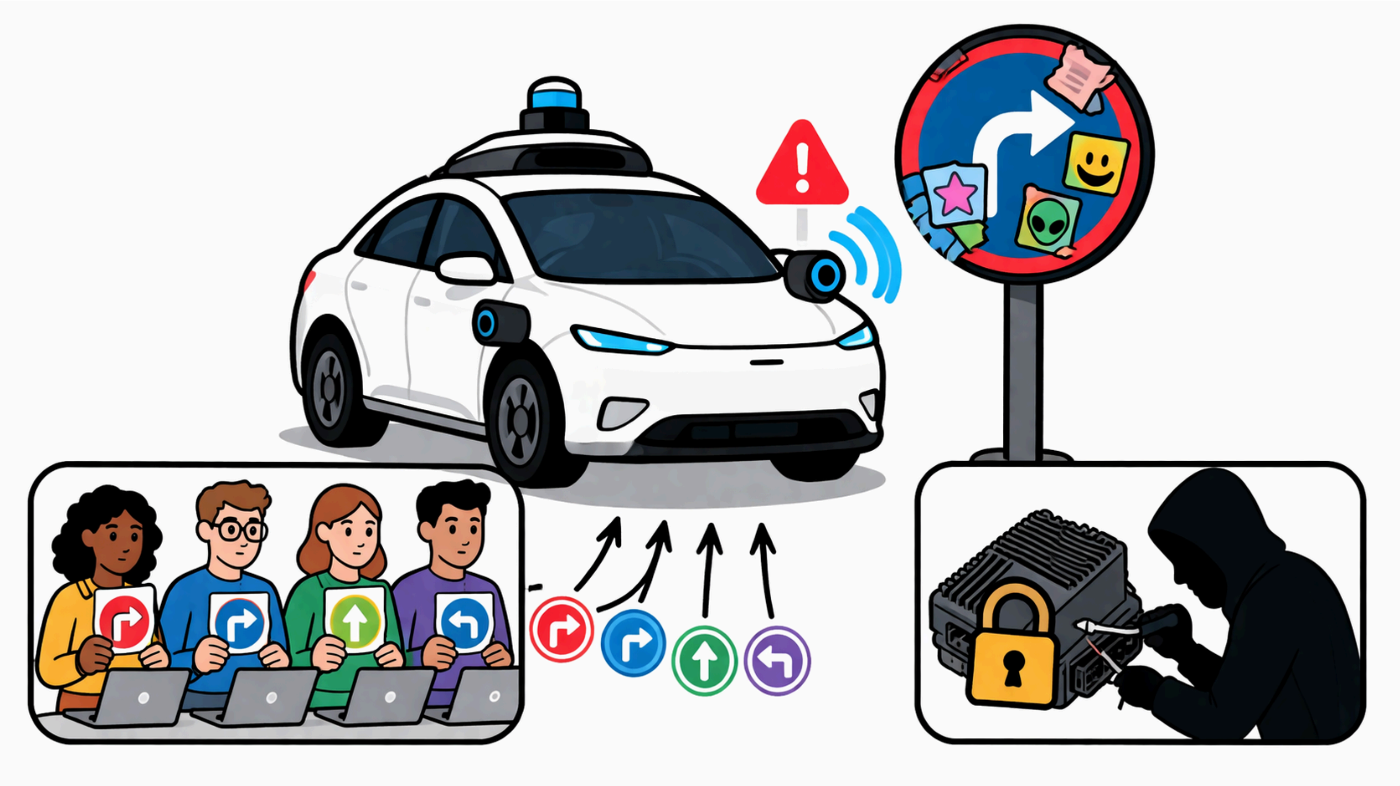
What really matters

- ML systems introduce **new attack surfaces** (training + inference)
- Security is **data-dependent**, not only system-dependent
- Privacy risks persist even without direct data access

What we need to study

- Trustworthy AI (adversarial ML, poisoning, evasion, inference attacks)
- Differential Privacy
- Federated Learning security
- Evaluation under adversarial settings

Use Case #4: Security Analysis of an Autonomous Perception System



Scenario

A perception model for an **autonomous vehicle** is deployed on an embedded ECU (Electronic Control Unit) with:

- Limited amount of ~~4 GB~~ RAM
- Limited compute budget
- Infrequent software update capability

The system must remain reliable under:

- Sensor noise and weather variability
- Physical-world adversarial manipulation (e.g., adversarial stickers)
- Supply-chain risks in the annotation pipeline

System Architecture

Function: Traffic sign recognition and scene perception

- Traffic sign recognition: the automatic recognition of traffic signs. The model must instantly understand what the sign says so that the car can react accordingly.
- Scene perception: a general understanding of the surrounding environment.

Deployment Paradigm: Edge inference + selective cloud interaction

- Edge inference: The AI model performs calculations and makes decisions directly in the car, using the vehicle's local hardware (an ECU with limited resources). Why is this done? For safety and speed reasons. A car traveling at 100 km/h cannot send camera footage to the cloud and wait for a response; the calculation must take place in fractions of a millisecond directly on-site (real-time constraint).
- Selective cloud interaction: The car is not completely isolated, but communicates with the central server (the cloud) in a targeted and non-continuous manner. What does it send to the cloud? Telemetry data, diagnostic system logs, or fault reports. What does it receive from the cloud? Software updates or updated and improved versions of the AI model (an operation that, however, occurs rarely precisely because of the system's limitations).

System Architecture

Data Pipeline

- Fleet-wide sensor data collection
- Annotation performed by three third-party vendors
- Centralized training performed in the vendor data center

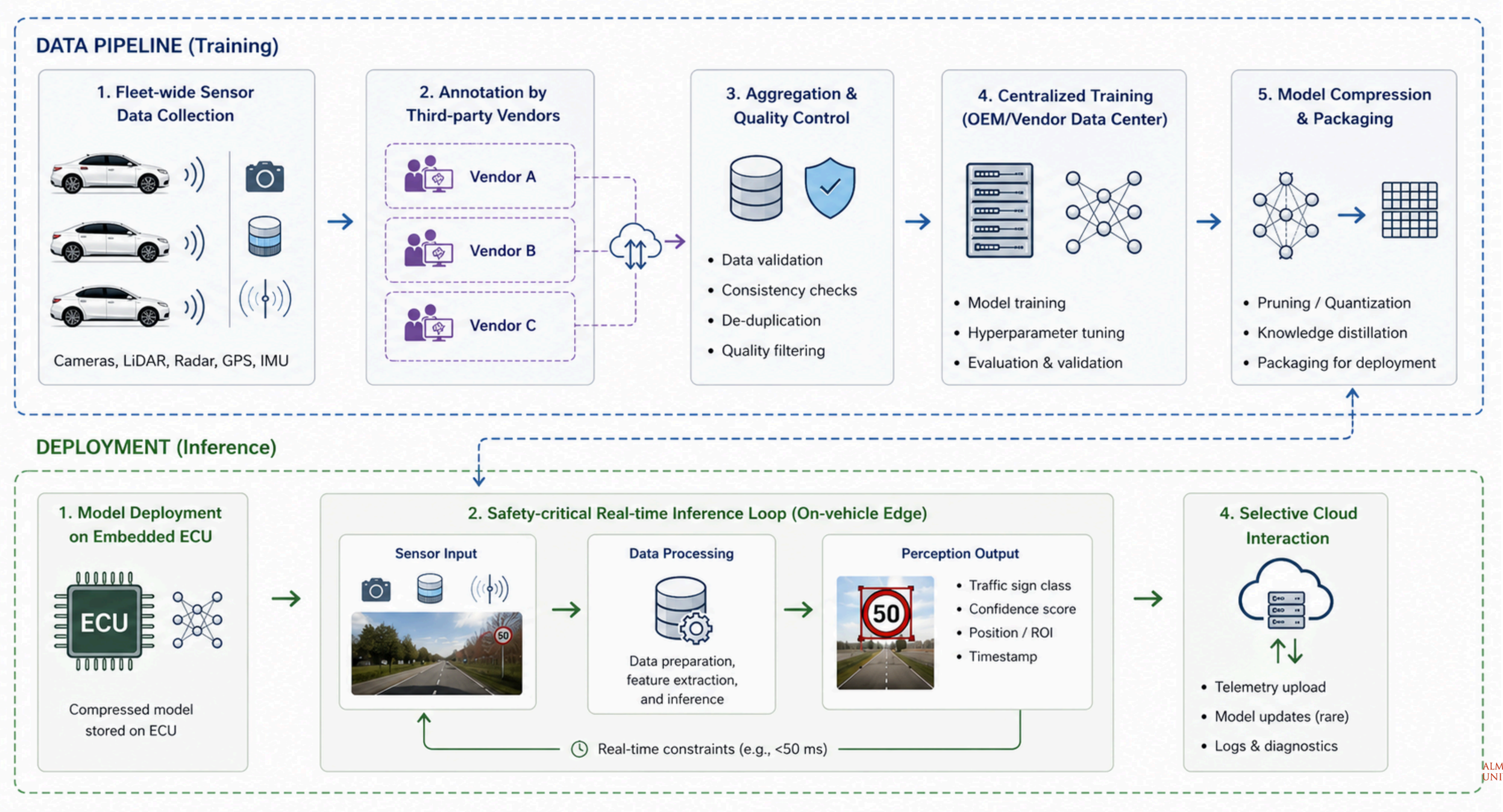
- Fleet-wide sensor data collection: Thousands of cars from the same automaker continuously collect data through their sensors as they drive normally. This raw data is sent to the parent company to be used as training material for AI.
- Annotation performed by three third-party vendors: Raw data is useless unless it is labelled to indicate what they represent. The task of "labelling" the images is outsourced to three external companies. This is a major weakness in the supply chain. A disloyal employee or an attacker infiltrating one of these external companies could intentionally alter the labels to poison the AI (Data Poisoning). This is why the slides mention the need to use three vendors for cross-checking.
- Centralized training performed in the vendor data center: Once the data is cleaned and labelled, the actual training of the model takes place centrally within a powerful data center.

Deployment

- Compressed model deployed on embedded ECU
- **No** continuous online retraining
- Safety-critical real-time inference loop

- Compressed model deployed on an embedded ECU: Deep learning models trained in data centers are enormous and require extremely powerful computers. The car's ECU has very limited resources. Consequently, the model is "downsized" and lightened before being installed in the vehicle's flash memory. The firmware stored in the ECU's flash memory is vulnerable to tampering attacks; this is why hardware defences such as Secure Boot are necessary.
- No continuous online retraining: The model installed on the car is static. The car does not learn new things or modify its algorithmic parameters while driving. Updates occur only rarely and through official channels controlled by the parent company.
- Safety-critical real-time inference loop: When the car is in motion, the camera sends a frame, the compressed model performs inference and decides what to do. This cycle must repeat continuously and extremely quickly. It is defined as safety-critical because even a minimal delay or a classification error can cause a fatal accident.

System Architecture



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Threat Modeling Framework

Security objectives:

- **Integrity**: Correctness of model parameters and outputs
- **Availability**: Real-time inference under adversarial conditions
- **Authenticity**: Assurance that deployed model is untampered

Integrity Threat 1

Physical Adversarial Patches

Attack Surface

Road signs modified with localized adversarial stickers

Integrity Threat 1

Threat ~~Model~~ Description

An adversary applies a perturbation δ such that:

$$\| \delta \|_\infty \leq \epsilon$$

The attacker wants the tampering to be invisible to the human eye (or to look like a harmless sticker or dirt on the sign), so as not to raise suspicion among passengers or human security guards.

but:

$$f(x) \neq f(x + \delta)$$

Even though to a human the road sign clearly remains a STOP sign (because the change delta is tiny), the AI becomes completely confused and misclassifies the image (for example, interpreting it as a 50 km/h Speed Limit sign).

Note: $\| x \|_\infty \; = \max \; | x_i |$

This rule extracts the maximum value of the change applied to each individual element or pixel.

Robustness-Capacity Trade-off

refers to the unavoidable engineering compromise between a machine learning model's security (robustness against adversarial attacks) and its performance and operational efficiency (computational capacity and baseline accuracy).

Certified robustness typically requires:

- Robustness to $\| \delta \|_\infty \leq \epsilon \rightarrow$ use of deep neural nets (Robustness vs. Model Size)

  To enforce stability over perturbation regions

- Deep NNs $\rightarrow$ longer training time (Robustness vs. Training Time)

  Robust training involves additional constraints and computations

- Overfitting avoidance $\rightarrow$ reduced accuracy on clean data (Robustness vs. Clean Accuracy)

  Smoother decision boundaries reduce sensitivity but also precision

To prevent an adversary from triggering a misclassification, robust training intentionally forces smoother decision boundaries. This desensitizes the model to malicious mathematical noise. By smoothing out these boundaries, the model loses its fine-grained precision. As a direct result, you suffer reduced accuracy on clean data. In a real-world driving scenario, a certified robust model will successfully ignore sneaky stickers on a stop sign, but it might become slightly worse at distinguishing normal, unmodified traffic signs under everyday conditions.

Integrity Threat 2

Annotation Supply-Chain Poisoning

The car will drive normally 80% of the time, but it will systematically fail to recognize certain critical signs that have been tampered with, ignoring, for example, a no-entry sign.

The most serious danger. The model passes all factory safety tests because no one notices the problem. However, an attacker (who knows the secret) can go to a real road, stick the trigger (the yellow sticker from the previous example) on a STOP sign, and deliberately cause a malfunction and an accident involving all the cars in the fleet passing by.

Attack Surface

Labels provided by three external vendors

Threat Description

- Systematic label corruption
- Targeted backdoor triggers
- Class-conditional bias injection

The attacker deliberately changes the labels of specific images. For example, they take hundreds of photos of STOP signs and label them as Speed Limit 50. During training, the AI will become confused and learn the wrong rule.

This is the most sophisticated threat. The attacker takes an image of a traffic sign, places a small, insignificant symbol (a yellow square, a Post-it note, a sticker) in the corner (trigger), and changes the label to Go. The trick: If the trigger isn't there, the car recognizes the STOP sign normally, but if the car encounters a real sign with that exact yellow sticker, the backdoor is activated and the car will drive straight through.

The attacker alters the data to ensure that the model develops a hidden mathematical bias only under certain conditions (for example, by altering the sign labels only in photos taken at night or in the rain), making the system vulnerable only in specific scenarios.

Potential consequences

- Misclassification of specific traffic signs
- Hidden backdoor activation

Integrity Threat 2: Mitigation

Controls

1. **Redundant Annotation**
    - Each sample labeled by ≥ 3 annotators
2. **Agreement Scoring**
    - Multi-rater consistency Inter-Rater Reliability (IRR) Scoring
    - Reject samples with consistent labels below threshold
3. **Vendor Drift Monitoring**
    - Per-vendor confusion matrix tracking
    - Statistical deviation detection

- The system creates and constantly updates a confusion matrix for each individual vendor. This matrix shows how many and which types of errors that vendor makes (e.g., how many times it misinterprets a STOP sign as a speed limit).
- If a vendor's performance or error patterns suddenly change over time in a way that deviates from the other two (creating a "statistical drift"), an alarm is triggered. For example, if Vendor A has always correctly identified 98% of signs and suddenly begins misidentifying 15% of STOP signs as "Go," the monitoring system detects a statistical anomaly and signals that that vendor's supply chain may have been compromised.

Integrity Threat 2

Trade-off

- Increased annotation cost
- Dataset size reduction
- Increased preparation latency

Availability Threat

Adversarial Sensor Inputs

Attack Surface

The attack surface is the real-time physical input that feeds the car's AI. An attacker can manipulate the real world by projecting laser lights, using projectors, or placing complex geometric patterns on the road to alter what the camera and LiDAR sensor (laser radar) capture and send to the onboard computer.

Camera and LiDAR streams

Threat Description

Inputs crafted to:

- Trigger worst-case compute paths
- Induce latency spikes
- Cause frame drops beyond control deadlines

- Many computer vision and 3D tracking algorithms work harder when the scene is chaotic or full of ambiguous geometric elements. The attacker creates an input that forces the software to perform the maximum possible number of loops, branches, and mathematical calculations, instantly saturating the ECU's CPU/GPU.
- Due to the computational overload described above, the time required for the AI to process a single frame skyrockets (from a few milliseconds to hundreds of milliseconds).
- Autonomous driving systems have strict deadlines (control deadlines, e.g., analyzing and responding to a frame within 50 ms). If the computation takes too long, the system is forced to "skip" or discard subsequent frames (frame drops) because it cannot keep up with the real-time video stream.

Consequence:

- The car can no longer process data at the same speed as it moves through physical space. It becomes temporarily " blind" to what is happening in its surroundings.
- If the onboard computer realizes it cannot meet the processing deadlines, it triggers an emergency procedure to shut itself down and instantly return control of the steering and brakes to the human driver. However, if this transition occurs suddenly, in the middle of a dangerous situation and without proper warning, the human has no time to react, causing an almost certain accident.

- Missed real-time constraints
- Unsafe control transitions

Availability: Mitigation

Controls

1. Input validation and preprocessing constraints
2. Worst-case latency profiling
3. Resource isolation at process level

- Before feeding the camera image or LiDAR data into the deep learning model (which is the computationally intensive part), the system performs a quick preliminary check and standardizes the data. If the input violates certain structural safety constraints, it is blocked or cleaned instantly.
- During the development phase, engineers test the software by stressing it with the worst possible inputs to map out maximum computation times. Based on this profile, safety timers (timeouts) are embedded in the car's code: if the vision algorithm is taking more than, tot time on a single frame, the calculation is forcibly truncated to prevent it from freezing the entire system.
- Many different programs run on the car's computer (brakes, steering, navigation, vision AI). This control ensures that the AI process runs in an isolated software cage, with a maximum limit on allocated RAM and computing power (e.g., up to 40% of the CPU). If the AI is attacked and reaches 100% of its allowed load, the slowdown remains confined there and does not affect the vehicle's vital processes, preventing the DoS attack from locking up the brakes or steering.

Trade-off:

- False rejection of legitimate rare scenarios

Metrics

- Maximum inference latency (ms)
- 99.9th percentile latency under adversarial load
- Frame drop rate

- The absolute maximum time recorded by the vehicle to process a single frame. It must strictly remain below the critical safety threshold (the control deadline).
- This is a crucial statistical metric. It involves sorting response times from fastest to slowest and identifying the value below which 99.9% of requests fall while the system is under attack. If even 99.9% of the frames are processed in time, it means the system is extremely stable and that the attack is failing.
- The percentage of video frames that the car is forced to discard because it cannot process them in time. The closer this percentage is to zero, the more reliable and responsive the system is.

Authenticity Threat

Model Tampering on ECU

Attack Surface

Model binary stored in on-device flash memory

Threat Description

- Unauthorized firmware modification
- Parameter replacement
- Backdoored model injection

Authenticity Mitigation: Trusted Execution

Controls

1. **Secure Boot**
   - SHA-256 integrity check of firmware at load time
2. **Trusted Execution Environment (TEE)**
   - Isolated inference runtime
   - Remote attestation

Metrics

- Integrity verification

Trade-off

- Increased startup latency
- Hardware cost overhead

Lessons Learned (Autonomous Systems)

What really matters

- The physical world becomes an **attack surface**
- Constraints (latency, memory) limit defenses
- Supply chain is part of the threat model

What we need to study

- Robustness of ML models
- Secure deployment (TEE, secure boot)
- Real-time constraints and worst-case analysis
- Data pipeline security

Use Case #5: Overlay Networks of Interacting AI Agents

Scenario

We consider a system composed of:

- N AI agents
- Each agent runs a Small Language Model (SLM)
- Agents interact over an overlay graph $G = (V, E)$
  - Nodes V: AI agents
  - Edges E: communication channels

The system is **fully decentralized**

- System behavior is no longer local, but **emergent and topology-dependent**

Why this is relevant

Future AI infrastructures will be:

- Distributed
- Autonomous
- Cooperative
- Heterogeneous

Examples:

- Personal agents interaction
- Distributed fraud detection
- Autonomous vehicle swarms
- Cyber threat intelligence sharing
- Edge IoT infrastructures

Why this is relevant

Agent Interaction Model

Each agent maintains a state s_i^t

Update rule:

$$s_i^{t+1} = F_i\left(s_i^t, \{s_j^t : j \in \mathcal{N}(i)\}\right)$$

where:

- $\mathcal{N}(i)$ = neighbors of agent i
- F_i implemented via SLM reasoning

Security implication

Local compromise → Global instability

Threat 1: Compromised High-Centrality Agent

Assume node k has:

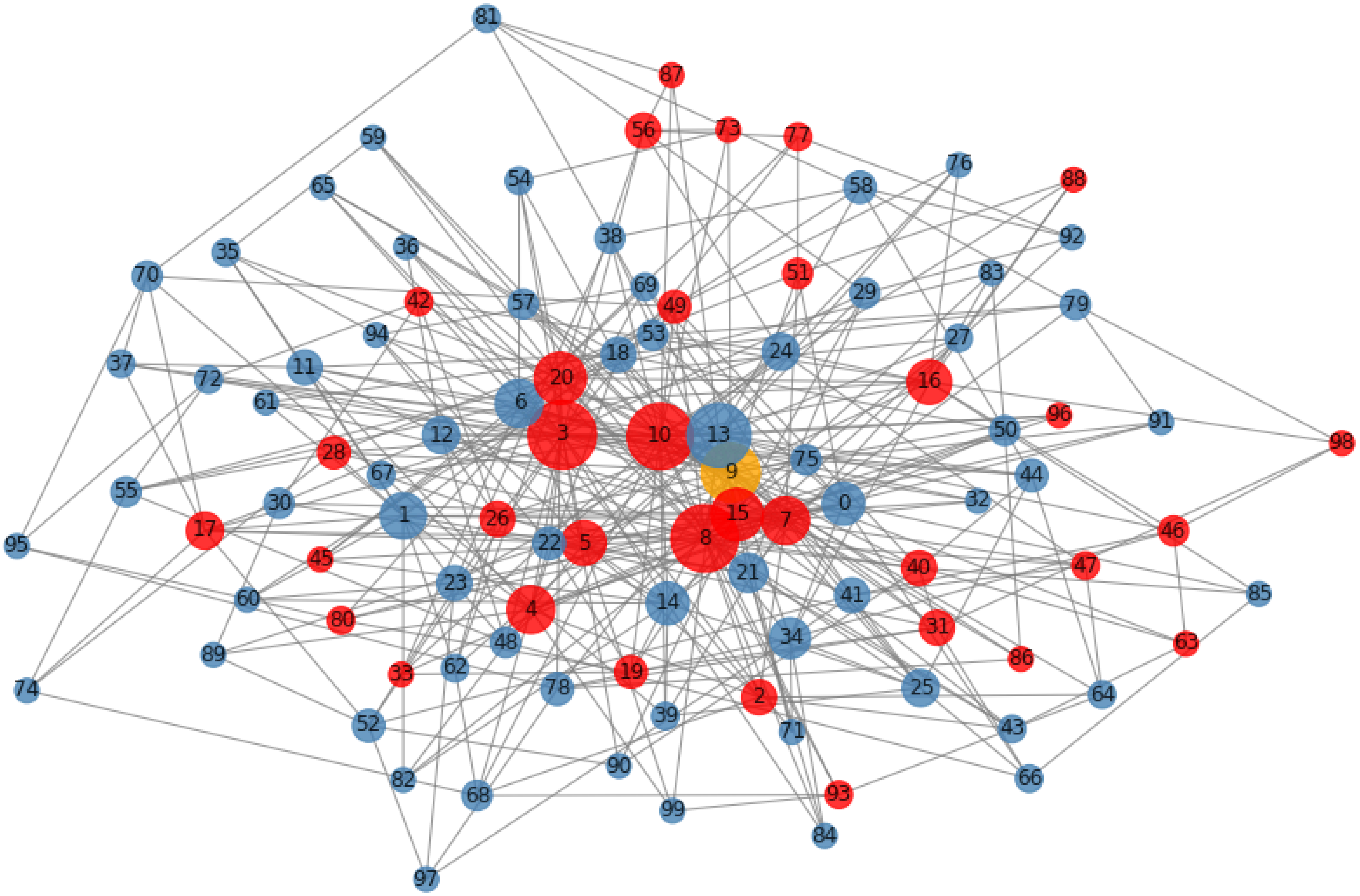
- High degree centrality
- High betweenness centrality
- High eigenvector centrality

If compromised: $s_k^t \rightarrow s_k^t + \delta$

Impact

- Rapid misinformation propagation
- Cascading behavioral drift
- System-wide instability

Threat 1: Compromised High-Centrality Agent



Threat 2: Coordinated Minority Attack

Subset $S \subset V$ behaves adversarially

Effects:

- Biased consensus
- Polarization
- Decision boundary shift

Critical question:

Is there a **phase transition threshold** for successful attack?

Threat 3: Topology-Driven Amplification

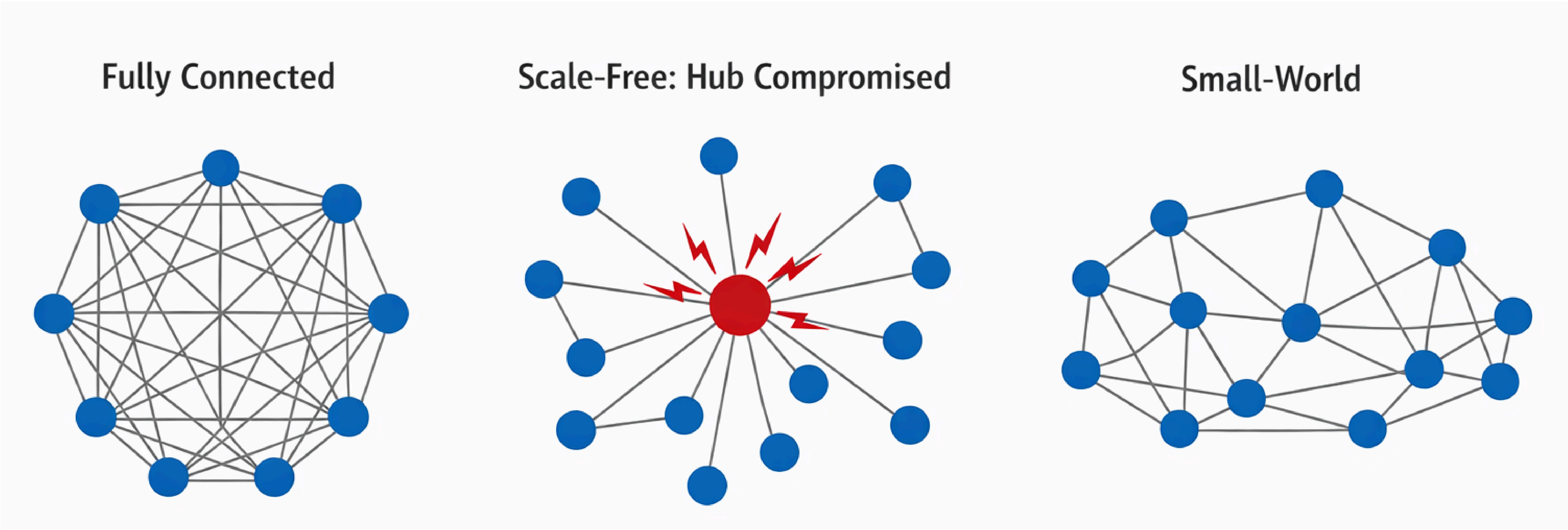
Certain graph structures amplify perturbations:

- Scale-free networks
- Highly clustered communities

Small perturbations may propagate exponentially

Security is topology-sensitive

Threat 3: Topology-Driven Amplification



Why Network Theory is Necessary

This is not a classical ML security problem, we are deadling with a **dynamical system over a graph**

We must analyze:

- Degree distribution
- Network connectivity
- Clustering coefficient
- Percolation thresholds

Mitigation 1: Trust-Weighted Aggregation

Modify update rule:

$$s_i^{t+1} = F_i\left(s_i^t, \sum_{j \in \mathcal{N}(i)} w_{ij} s_j^t\right)$$

Where:

- w_{ij} = trust weights
- dynamically updated

Goal: reduce adversarial influence

Mitigation 2: Topology Engineering

Design graph to improve robustness:

- Increase net connectivity
- Limit hub dominance
- Avoid extreme scale-free structures
- Introduce redundancy

In this scenario, security is partially a graph design problem

Mitigation 3: Byzantine-Resilient Consensus

Use robust aggregation:

- Median-based update
- Trimmed mean
- Outlier rejection

Goal: Bound adversarial impact even if fraction f of nodes is malicious

Mitigation 4: Influence Monitoring

Compute periodically:

- Node centrality metrics
- Influence functions
- Local divergence indicators

Early detection of:

- Behavioral drift
- Coordinated anomalies

Lessons Learned (AI Agent Networks)

What really matters

- Security is **emergent and topology-dependent**
- Local compromises can cause **global failures**
- Trust must be **distributed and dynamic**

What we need to study

- Network theory and graph robustness
- Distributed consensus under adversaries
- Trust and reputation systems
- Dynamical systems and stability analysis

Part III: Some problems and techniques we have identified so far

Foundational Terms - CIA

Foundational Terms - CIA + ML Extension

The classic **CIA triad** remains the backbone:

Property	Classic Definition	ML Extension
Confidentiality	Data visible only to authorized parties	Training data, model weights, inference outputs
Integrity	Data/system not modified without authorization	Model predictions not manipulated by adversary
Availability	System accessible when needed	Model not degraded by denial-of-service or poisoning

Threat Actors & Their Capabilities

We classify adversaries along two axes:

Knowledge:

- **White-box**: full access to model architecture, weights, training data
- **Gray-box**: partial knowledge (architecture known, weights unknown)
- **Black-box**: query-only access (most realistic in production)

Goals:

- **Integrity attacks**: cause misclassification
- **Privacy attacks**: extract sensitive information from model
- **Availability attacks**: degrade model performance
- **Backdoor attacks**: embed hidden behavior triggered by a specific input pattern

Threat Modeling for ML Systems

The ML pipeline as an attack surface:

```
[Data Collection] → [Preprocessing] → [Training] → [Validation] → [Deployment] → [Inference]
       ↑                   ↑              ↑                              ↑                ↑
  Poisoning          Label flipping    Model theft                Model inversion     Evasion
```

AI for Cybersecurity

to conclude ... since we gave it for granted

AI as a Tool for Defense Cybersecurity

Application	ML Approach	Key Challenge
Intrusion Detection	Anomaly detection, sequence models (e.g., LSTM)	Maintaining low false positives under high-volume traffic
Malware Classification	Static/dynamic analysis, GNNs	Robustness to adversarial evasion techniques
Fraud Detection	Supervised learning (e.g., gradient boosting), online learning	Drift due to evolving attacker adaptation
Behavioral Analytics	Sequence modeling, user embeddings	Preserving user privacy while modeling behavior
Threat Intelligence	NLP on CTI reports, knowledge graphs	Data quality and noise in OSINT

Some Recommended Resources to Start

Primary references:

- Goodfellow et al. — *Deep Learning* (MIT Press) — Ch. on Robustness
- Dwork & Roth — *The Algorithmic Foundations of Differential Privacy* (2014)
- Papernot et al. — *SoK: Security and Privacy in ML* (IEEE EuroS&P 2018)

Tools & frameworks:

- `CleverHans`, `ART (IBM)` — adversarial attack/defense libraries
- `Opacus` (PyTorch), `TF Privacy` — differential privacy
- `PySyft` — federated learning & SMPC
- `FATE` — industrial federated learning framework

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Prof. Stefano Ferretti

Department of Computer Science and Engineering

University of Bologna

s.ferretti@unibo.it